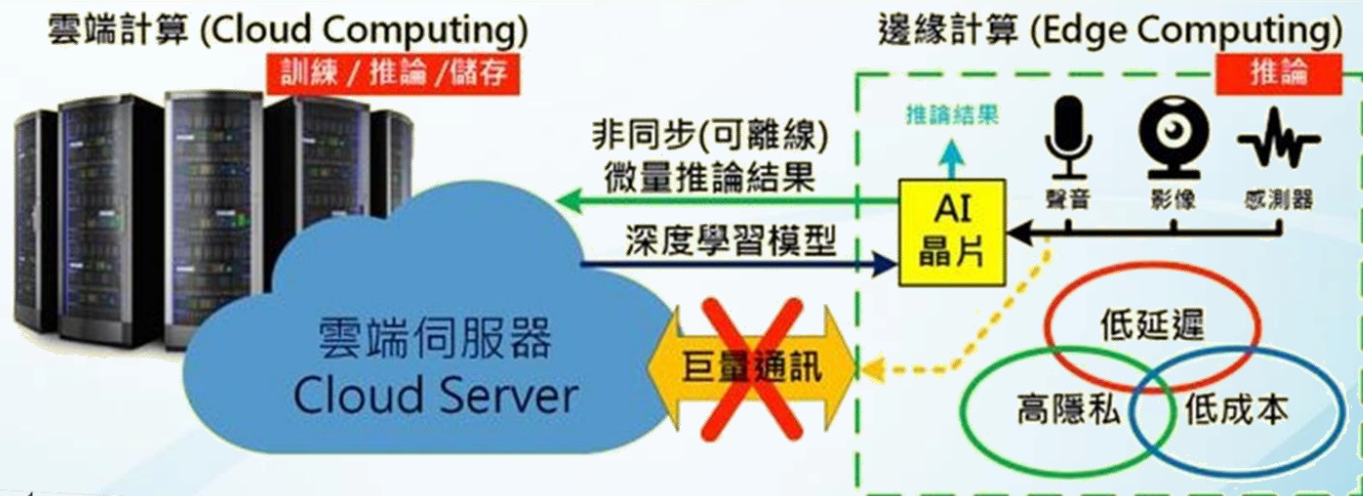
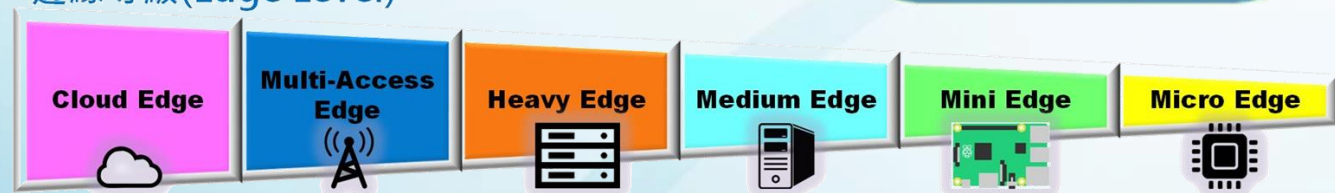


邊緣人工智慧實務 (EE5354701)



Edge AI

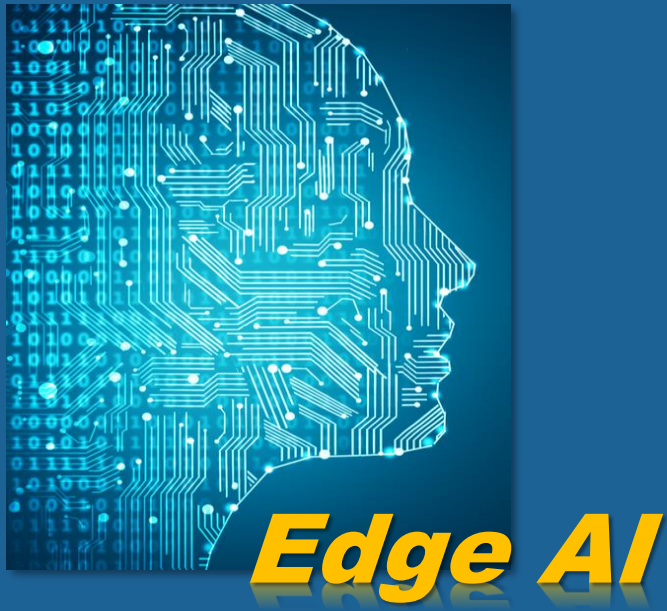
邊緣等級(Edge Level)



7 Edge AI 案例實作 — 影像分割、姿態估測

電機工程系 人工智慧應用產碩專班 許哲豪 兼任助理教授

簡報大綱

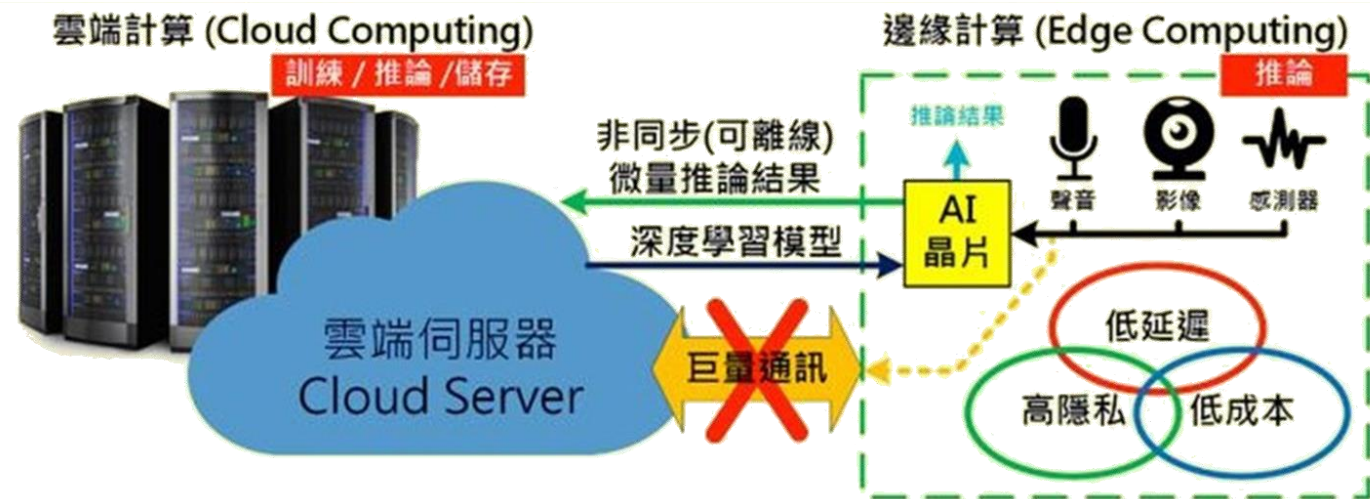


- 7.1. 影像分割
- 7.2. 姿態估測
- 7.3. Google MediaPipe

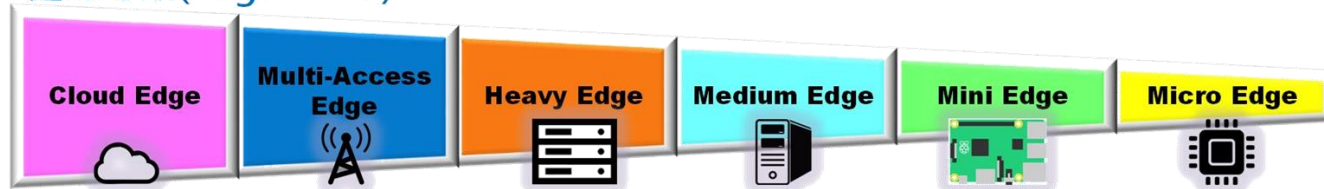
https://hackmd.io/@OmniXRI-Jack/NTUST_EdgeAI_2026



Edge AI

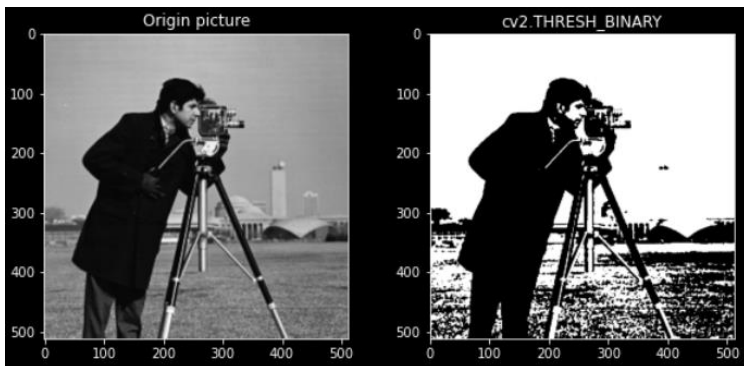


邊緣等級(Edge Level)



7.1. 影像分割

傳統影像分割技術 — 像素特徵



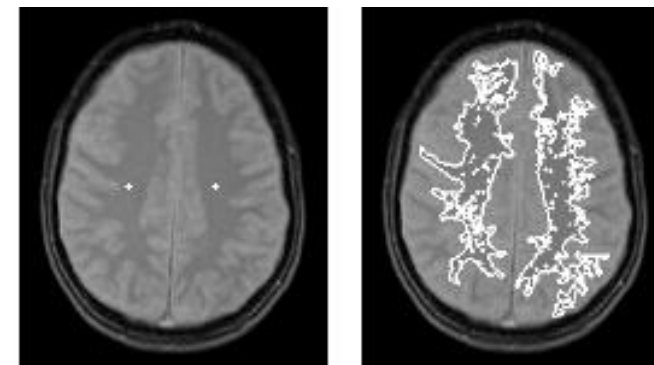
閾值處理 (Thresholding)

- 根據像素亮度或色彩差異設定門檻值，將其二值化。



邊緣偵測 (Edge Detection)

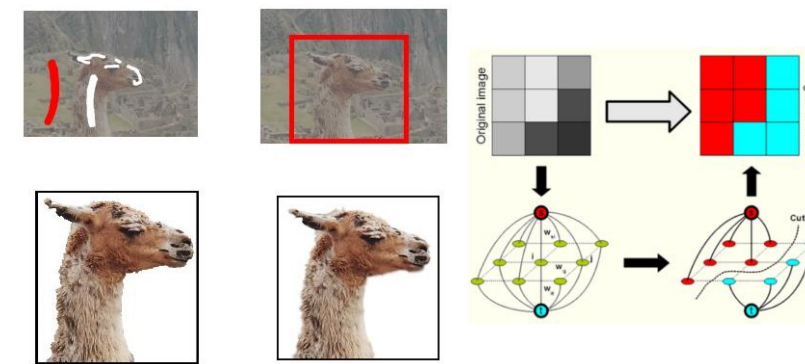
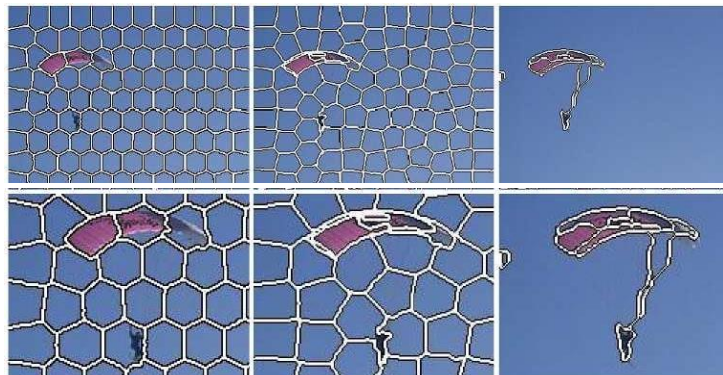
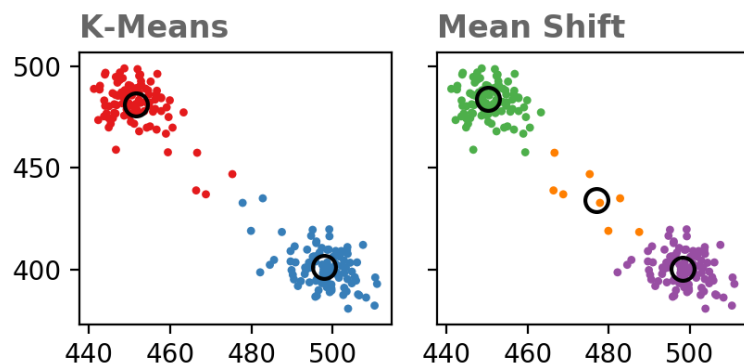
- 利用邊緣偵測算子（如 Sobel, Canny等）找出像素中亮度或色彩有明顯變化的邊界。



區域增長 (Region Growing)

- 從種子像素開始，找出相鄰像素亮度或色彩較相近的進行區域合併。
- 分水嶺法

傳統影像分割技術 — 聚類、圖論



像素聚類 (Clustering)

- **K-Mean**：指定K個分類，經過多輪迭代，找出聚中心及最靠近的像素。
- **Mean Shift**：不用指定分類數量，找出資料分佈局部密度最高區域。

超像素 (Super Pixel)

- **SLIC**：將具備類似特徵的像素集成超大像素，一開始可指定欲產出超像素數量。
- **SLICO, MSLIC ...**

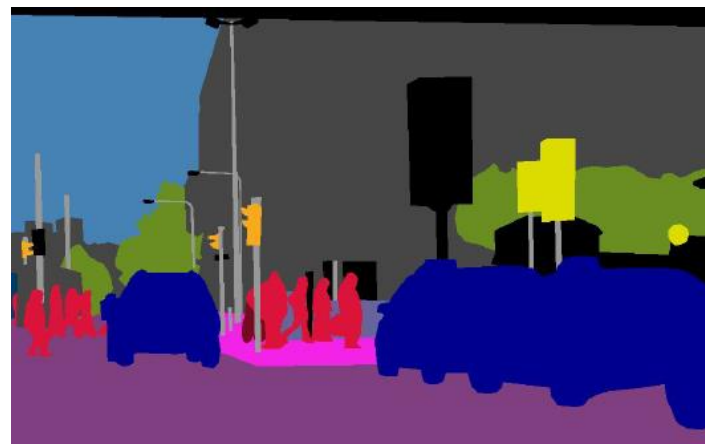
圖論式分割 (GrabCut)

- **互動式分割**，設定前景及背景，再以圖論方式找出合理邊界。分割不理想處可再增減前、背景設定。

AI 影像分割技術 — 像素級分割任務



原始影像



語義 (Semantic) 分割

相同分類像素
採用相同數值
(色彩) 輸出。

先找出物件框
再進行像素級
分割。



實例(Instance)分割



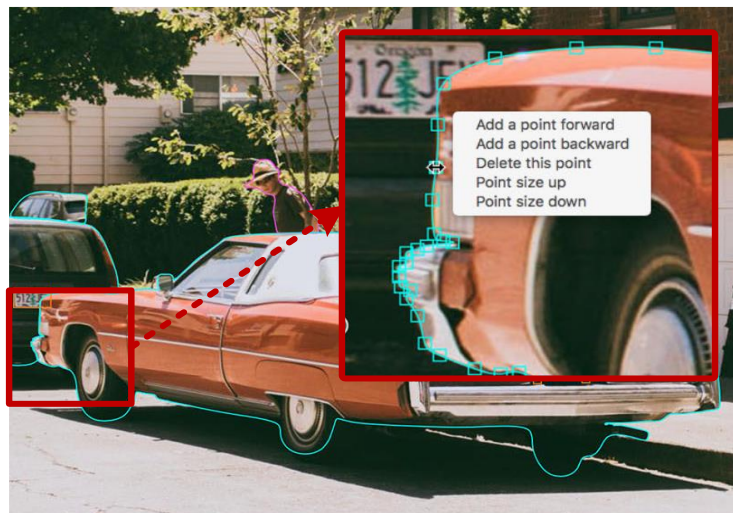
全景(Panoptic)分割

結合語義及實
例分割特性，
將所有像素都
進行分類。

AI 影像分割技術 — 影像分割標註方式

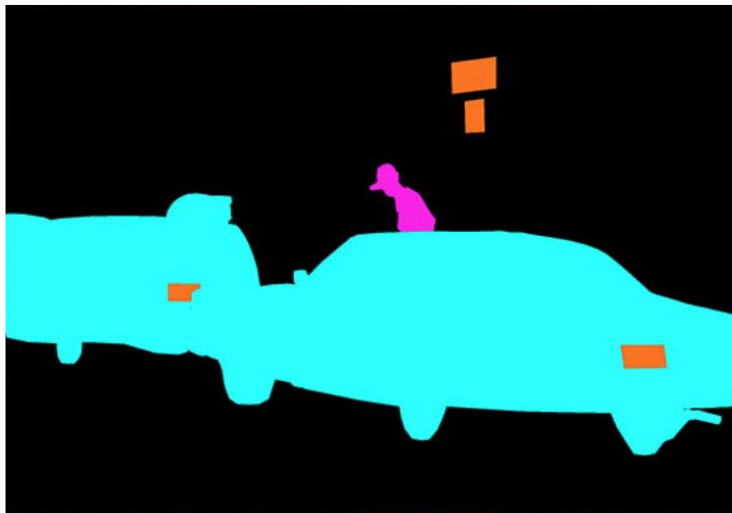
貝茲曲線
(Bezier)

不受影像
尺寸影響



封閉多邊形
(Polygon)

不精準但
標註速度快



自由筆刷
(Brush)

很不精準



超像素
(Superpixel)

OpenCV

- LSC
- SEEDS
- SLIC

無自動合併



AI 影像分割技術 — VOC 資料集格式



原始影像 (*.jpg)



語義分割後影像 (*.png)

黑色為背景

白色為邊界
(非必要)

PNG標註檔
採索引式
(Index)格式
類似GIF格式

Aeroplane	Diningtable	Bottle	Person	Chair	Train
Bicycle	Cat	Bus	Pottedplant	Cow	Tvmonitor
Bird	Horse	Car	Sheep		
Boat	Motorbike	Dog	Sofa		

VOC 20+1類別

PNG8 (Indexed, Palette)格式

4-color



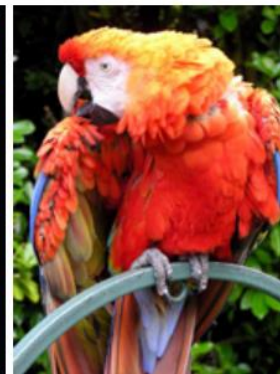
16-color



256-color

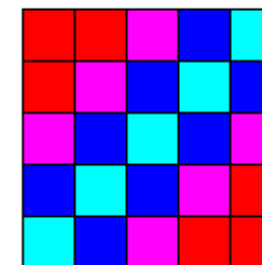


True color



0	0	1	2	3
0	1	2	3	2
1	2	3	2	1
2	3	2	1	0
3	2	1	0	0

0 = 
 1 = 
 2 = 
 3 =  indexed



PNG8使用8bit ($2^8 = 256$ 色)來表示索引色，
須搭配調色盤使用。

資料來源：<https://zh.wikipedia.org/zh-tw/PNG>，https://en.wikipedia.org/wiki/Indexed_color

AI 影像分割技術 — COCO資料集格式

COCO 資料格式

```
{
  "info": info,
  "images": [image],
  "annotations": [annotation],
  "licenses": [license],
}

info{
  "year": int,
  "version": str,
  "description": str,
  "contributor": str,
  "url": str,
  "date_created": datetime,
}

image{
  "id": int,
  "width": int,
  "height": int,
  "file_name": str,
  "license": int,
  "flickr_url": str,
  "coco_url": str,
  "date_captured": datetime,
}

license{
  "id": int,
  "name": str,
  "url": str,
}
```

影像分類/物件偵測/語義分割標註資料格式

```
annotation{
  "id": int,
  "image_id": int,
  "category_id": int,
  "segmentation": RLE or [polygon],
  "area": float,
  "iscrowd": 0 or 1,
  "bbox": [x,y,width,height],
}

categories[{
  "id": int,
  "name": str,
  "supercategory": str,
}]
```

Segmentation : [polygon]

```
"segmentation":
[[510.66,423.01,...,510.45,423.01]],
"area": 702.10574999999998,
"iscrowd": 0,
```

Segmentation : [RLE]

```
"segmentation": {"counts":
[20736,2,453,5,452,9,447,13,444,...,5,34552],
"size": [457,640]
},
"area": 3074,
"iscrowd": 1,
```

OmniXRI Oct. 2020 整理繪製

Segmentation : [polygon] 封閉多邊形

```
"segmentation": X1, Y1, X2,Y2, ... ,Xn, Yn
[[510.66,423.01,...,510.45,423.01]],
"area": 702.10574999999998,
"iscrowd": 0,
```

Segmentation : [RLE] 二進制影像壓縮

```
"segmentation": {"counts":
[20736,2,453,5,452,9,447,13,444,...,5,34552],
"size": [457,640]
},
"area": 3074,
"iscrowd": 1,
```

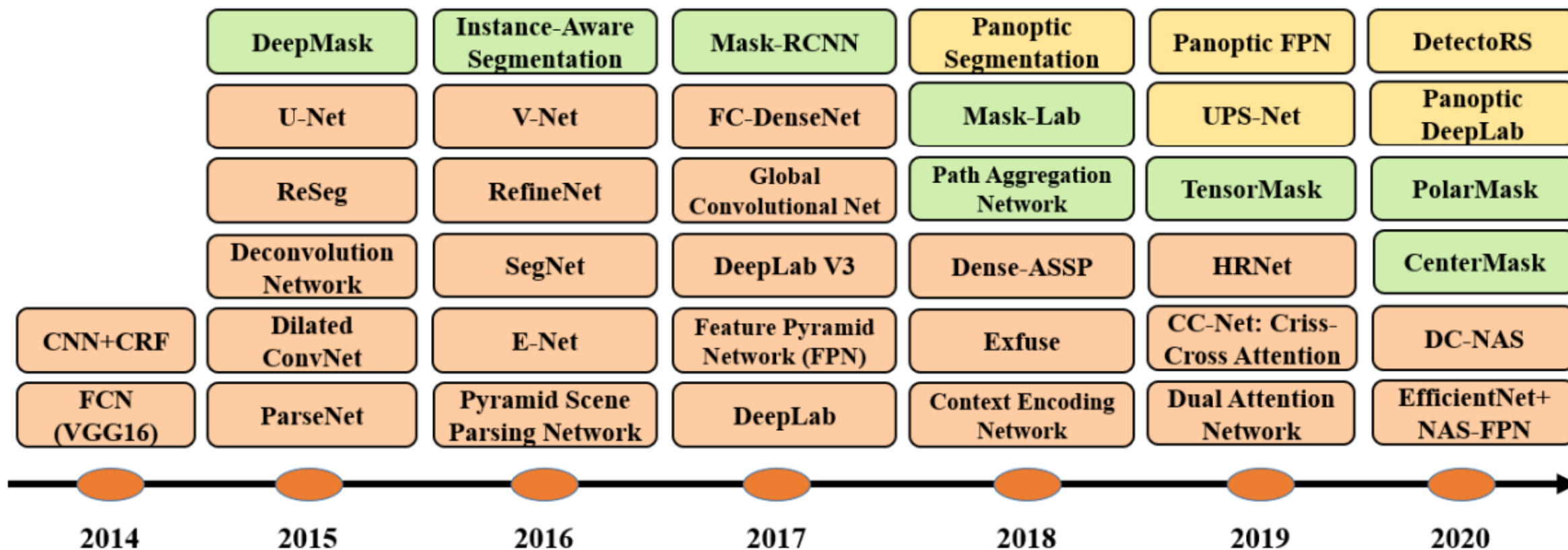
黑像素個數, 白像素個數, 黑, 白, ...

AI 影像分割技術 — 基於卷積神經網路

橙色：語義分割

綠色：實例分割

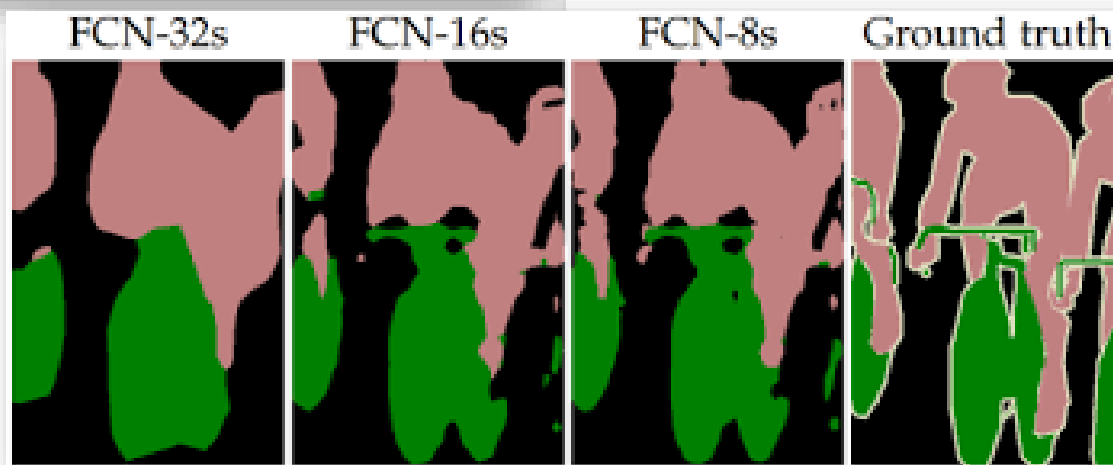
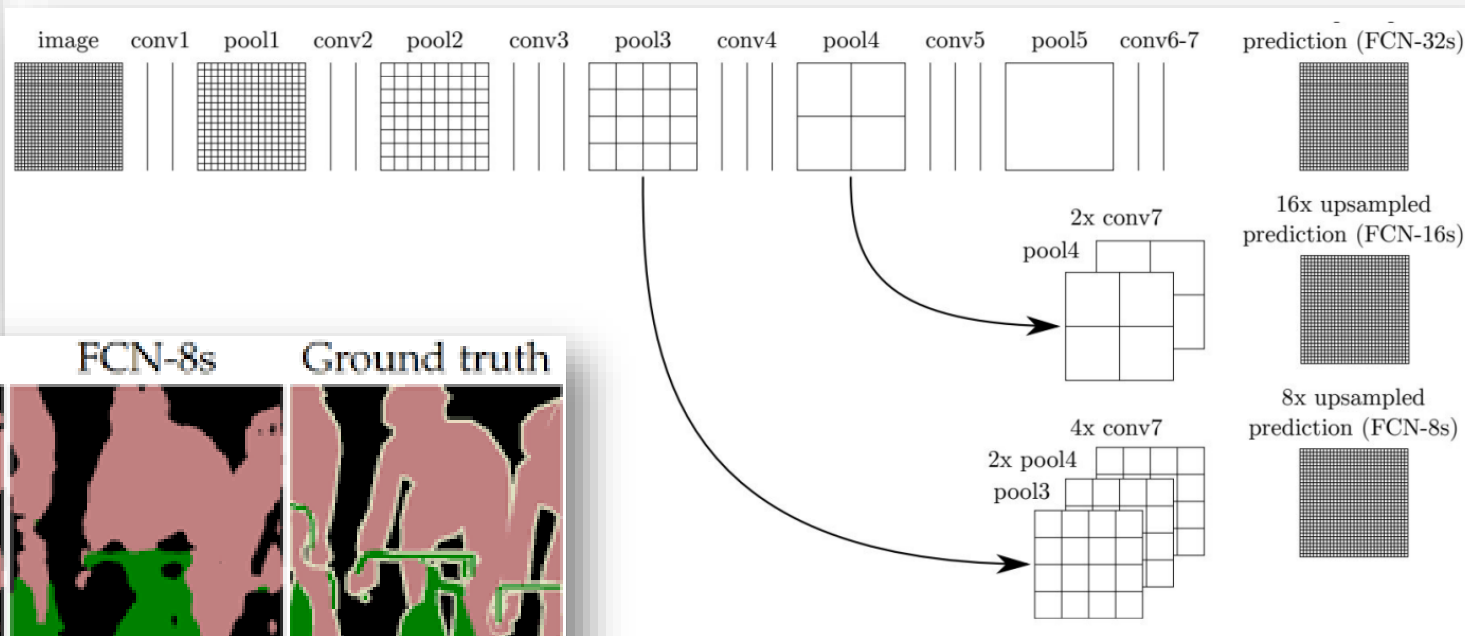
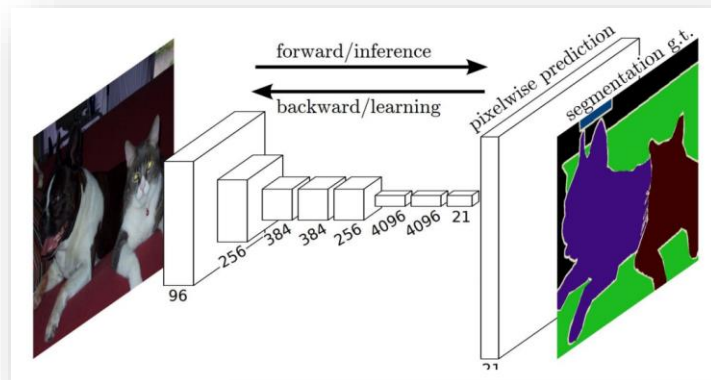
黃色：全景分割



參考資料：<https://arxiv.org/abs/2001.05566>

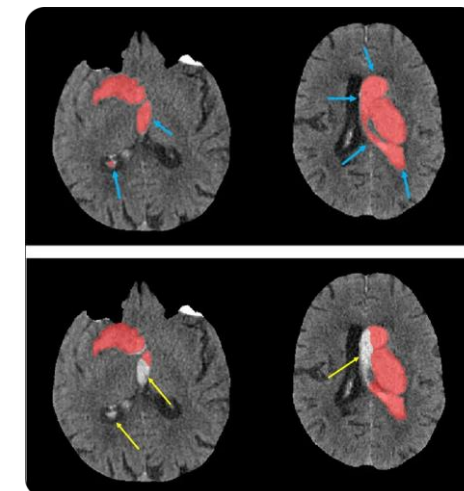
影像語義分割技術 — FCN

Fully Convolutional Networks Based Model

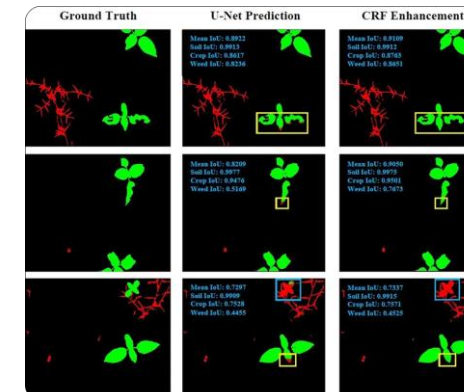


參考資料：<https://arxiv.org/abs/1411.4038>

醫療應用

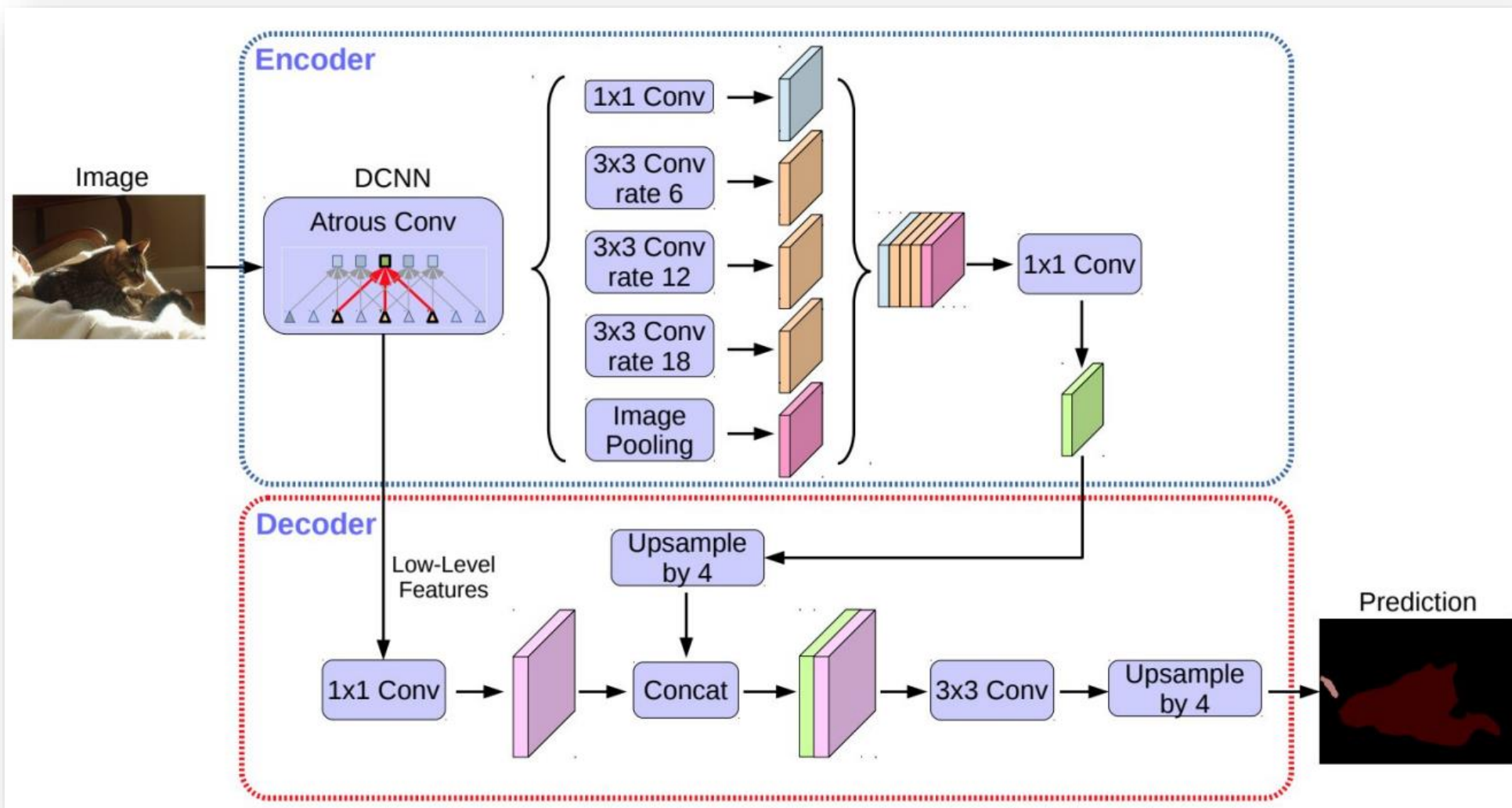


農業應用



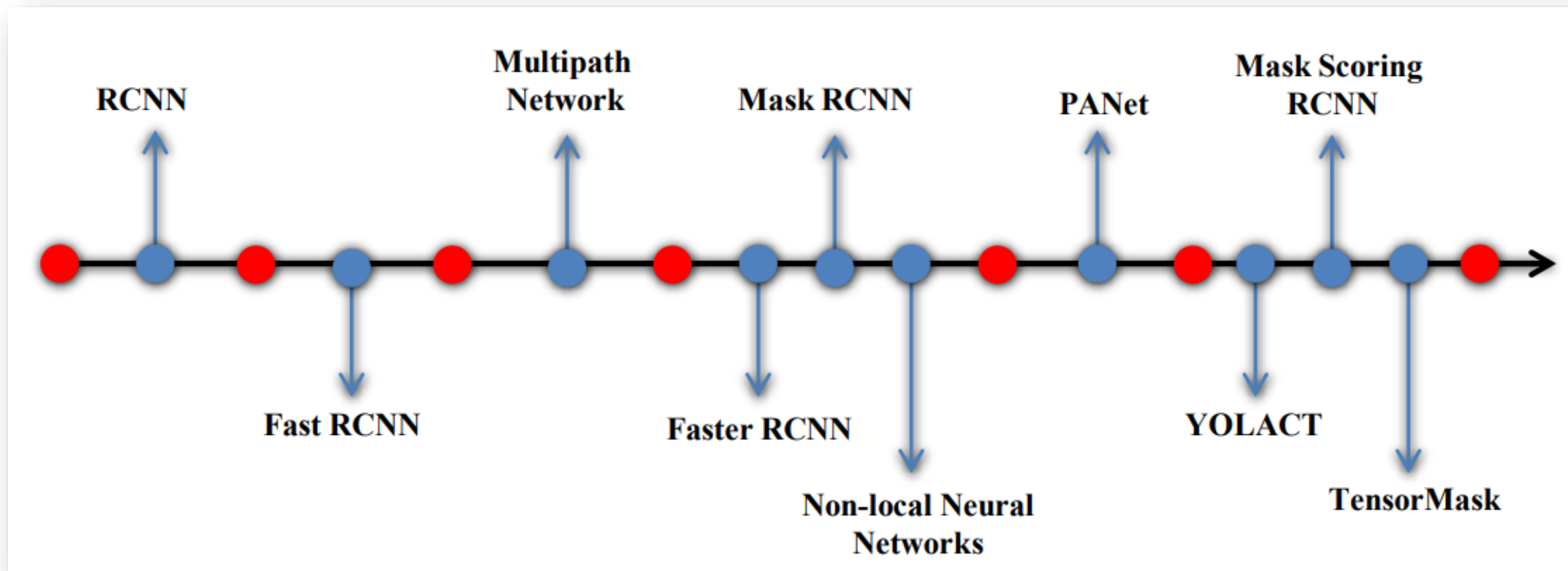
2026/04/27

影像語義分割技術 — DeepLabV3+



參考資料：<https://arxiv.org/abs/1802.02611>

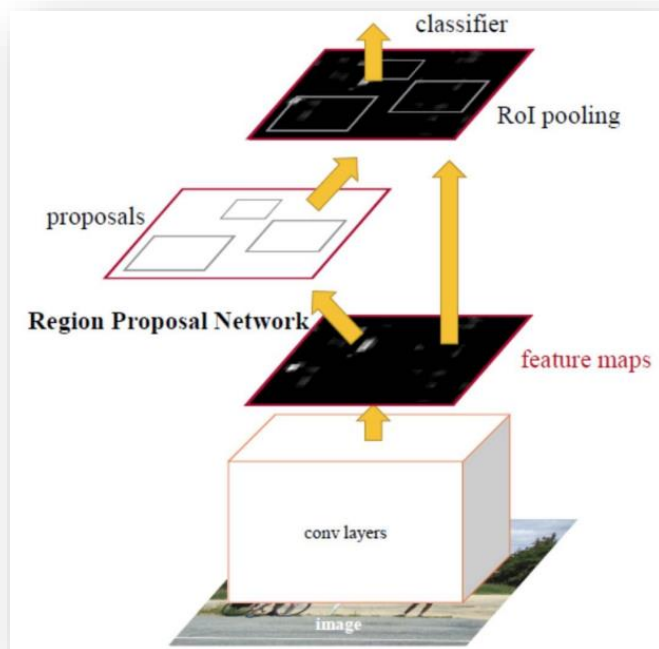
AI 影像分割技術 — 實例分割技術演進



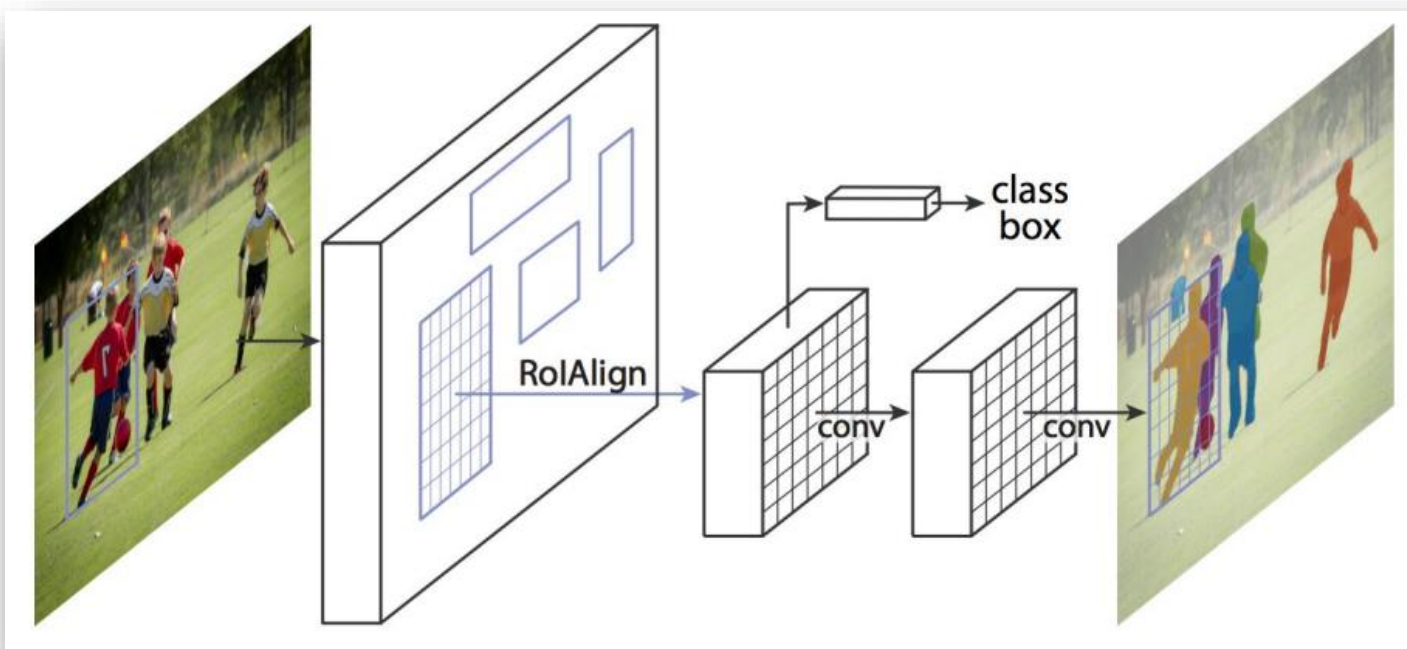
資料來源：<https://arxiv.org/abs/2007.00047>

影像實例分割技術 — Mask R-CNN

Region Convolution Neural Network Based Model



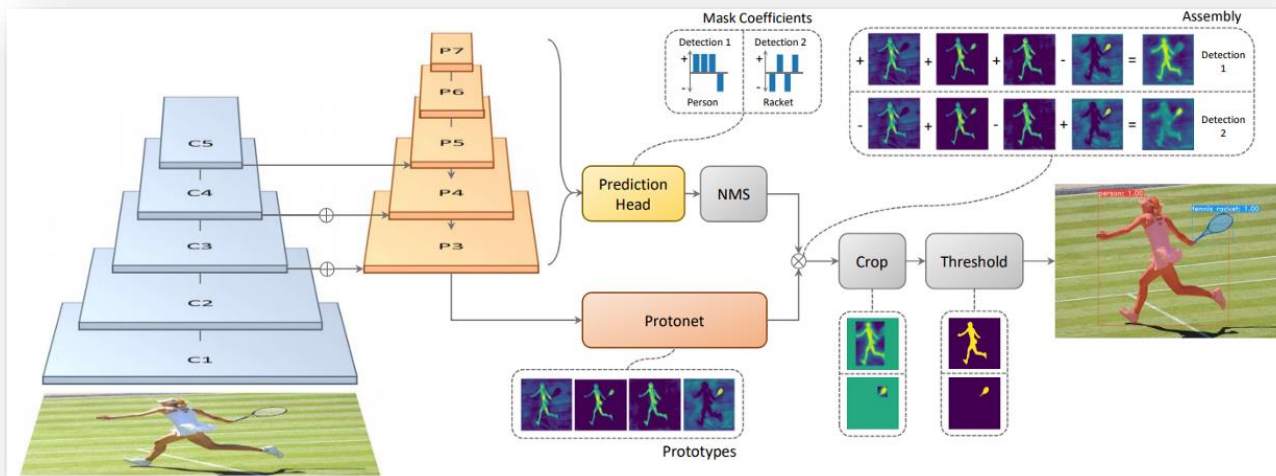
Faster R-CNN



Mask R-CNN

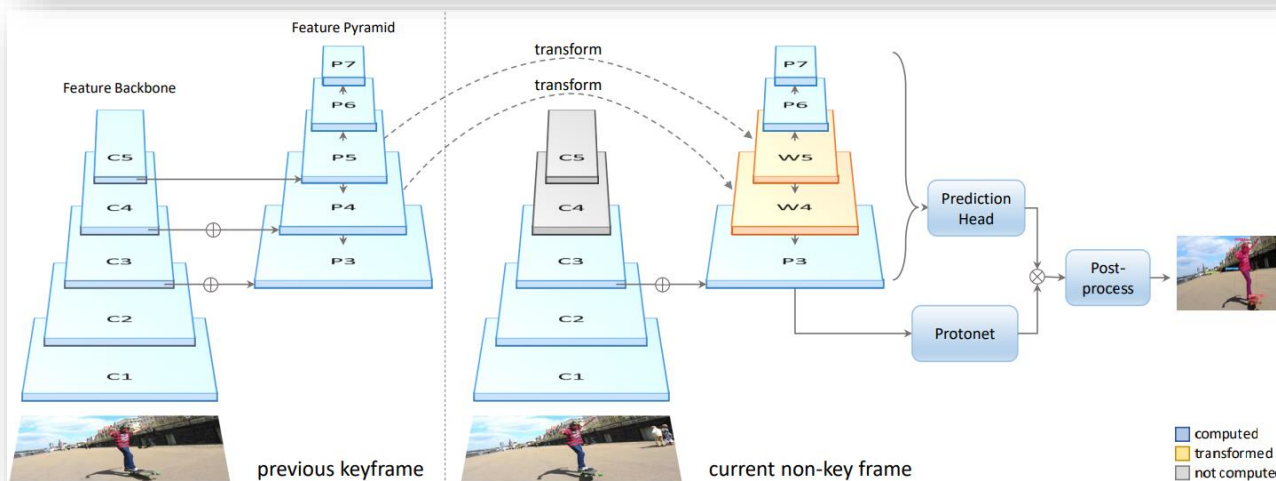
參考資料：<https://arxiv.org/abs/1703.06870>

影像實例分割技術 — Yolact & Edge



You Only
Look At
Coefficients

<https://arxiv.org/abs/1904.02689>

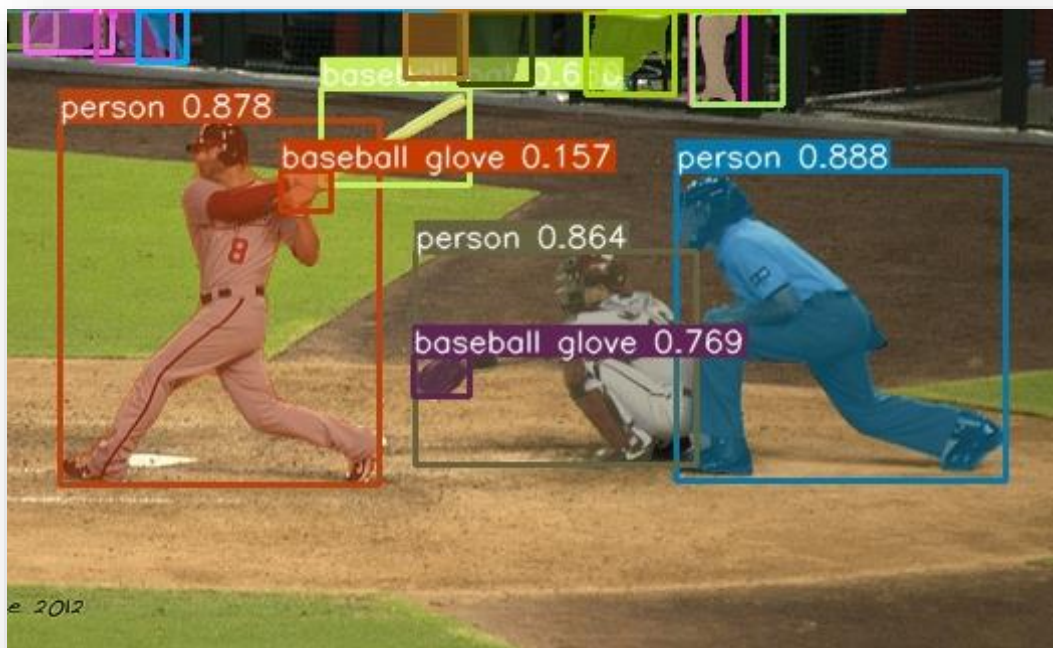


YOLACT
Edge

<https://arxiv.org/abs/2012.12259>

影像實例分割技術 — YOLO 系列

中研院 (Academia Sinica)
YOLOv7



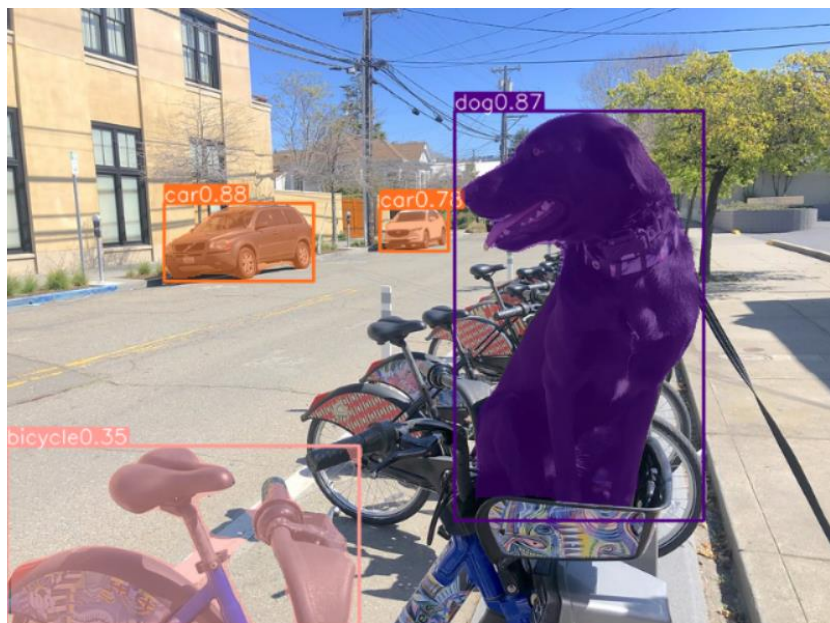
<https://github.com/WongKinYiu/yolov7>

Ultralytics
YOLO v8 / v11 / 26



<https://docs.ultralytics.com/tasks/segment/>

Intel OpenVINO YOLO 實例分割範例



Ultralytics
YOLO v8 / v11 / 26 / E-26



AI Trends Convert and Optimize YOLO26 instance segmentation model with OpenVINO™

Model Demos • Object Detection • Image Segmentation

[View on GitHub](#)
[Open in Colab](#)
[Show Status](#)
[Link](#)



AI Trends YOLOE-26 Open-Vocabulary Detection and Segmentation with OpenVINO™

Model Demos • Object Detection • Image Segmentation • Zero-Shot Image Classification

[View on GitHub](#)
[Open in Colab](#)
[Show Status](#)
[Link](#)



AI Trends Convert and Optimize YOLOv11 instance segmentation model with OpenVINO™

Model Demos • Object Detection • Image Segmentation

[View on GitHub](#)
[Open in Colab](#)
[Show Status](#)
[Link](#)



AI Trends Convert and Optimize YOLOv8 instance segmentation model with OpenVINO™

Model Demos • Object Detection • Image Segmentation

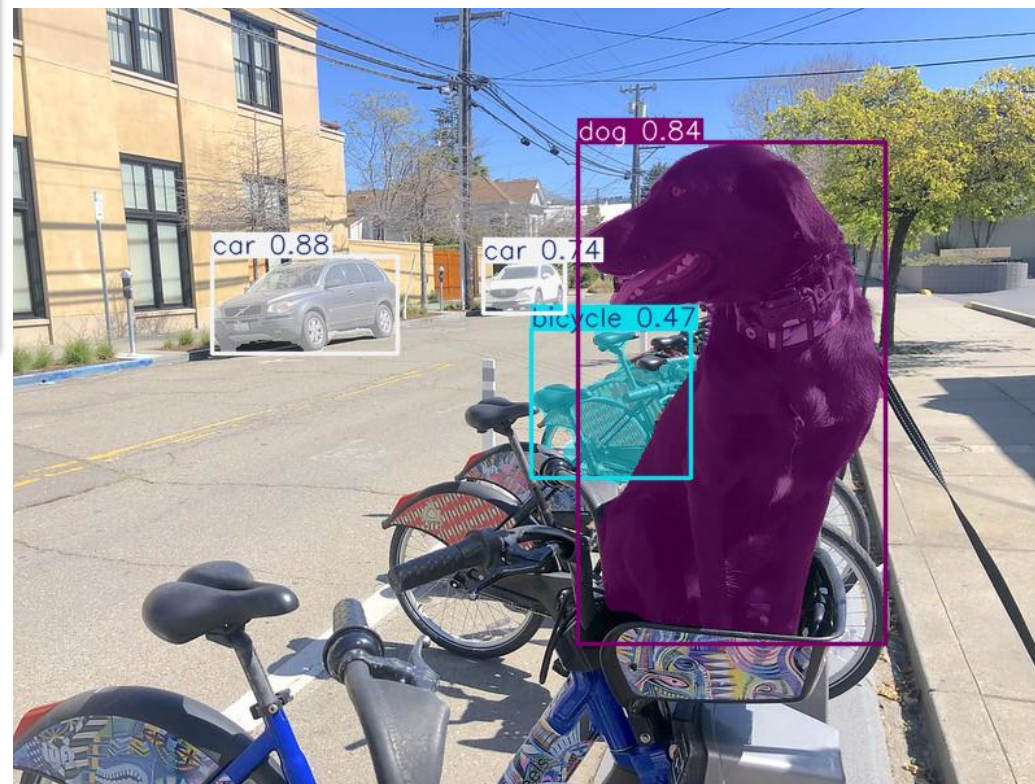
[View on GitHub](#)
[Open in Colab](#)
[Show Status](#)
[Link](#)

https://openvinotoolkit.github.io/openvino_notebooks/?search=YOLO

範例7-1：OpenVINO YOLO26 實例分割

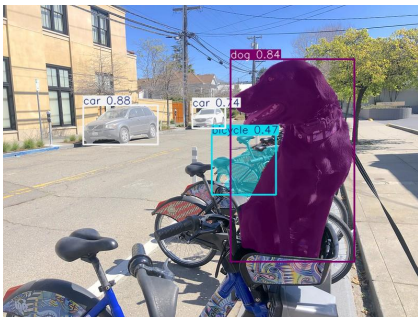
- [Get PyTorch model](#)
 - [Prerequisites](#)
- [Instantiate model](#)
 - [Convert model to OpenVINO IR](#)
 - [Verify model inference](#)
 - [Select inference device](#)
 - [Test on single image](#)
- [Optimize model using NNCF Post-training Quantization API](#)
 - [Validate Quantized model inference](#)
- [Compare performance of the Original and Quantized Models](#)
- [Other ways to optimize model](#)
- [Live demo](#)
 - [Run Live Object Detection and Segmentation](#)

可支援推論模型
yolo26n-seg
Yolo26s-seg
yolo26m-seg
yolo26l-seg
yolo26x-seg



說明文檔：https://github.com/openvinotoolkit/openvino_notebooks/tree/latest/notebooks/yolov26-optimization
範例程式：https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-instance-segmentation.ipynb

Yolo26 不同模型實例分割時間比較



前處理：影像色彩、影像尺寸、數值正規化 ...

推論：依選擇模型進行物件偵測推論

後處理：繪製物件框、標籤、置信度、**著色** ...

Ps. 雲端推論會視網路及伺服器負擔每次推論時間會有很大不同

Model Format	Preprocess	Inference	Postprocess
Yolo26 PyTorch	34.9 ms	882.5 ms	22.8 ms
OpenVINO IR	9.1 ms	404.1 ms	14.8 ms
OpenVINO NNCF	8.8 ms	180.2 ms	13.7 ms

20.4%
(4.90x)

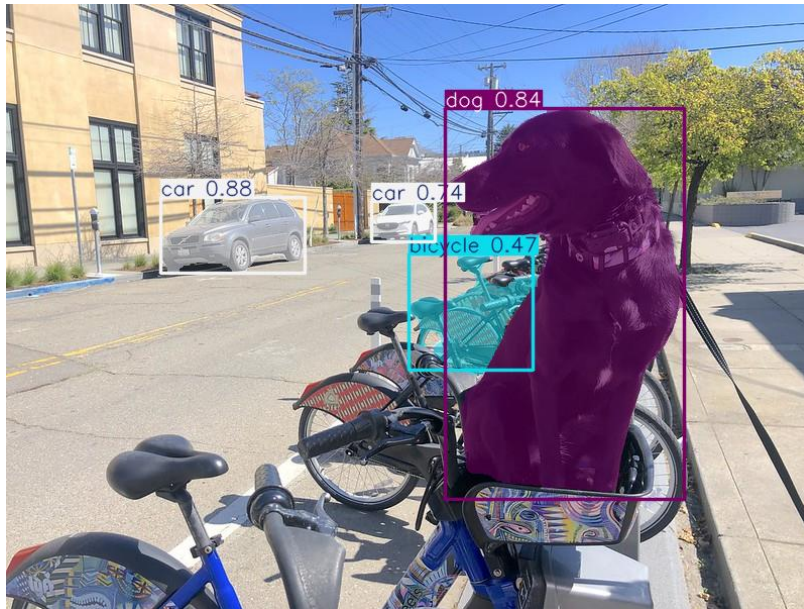
45.8%
(2.18x)

44.6%
(2.24x)

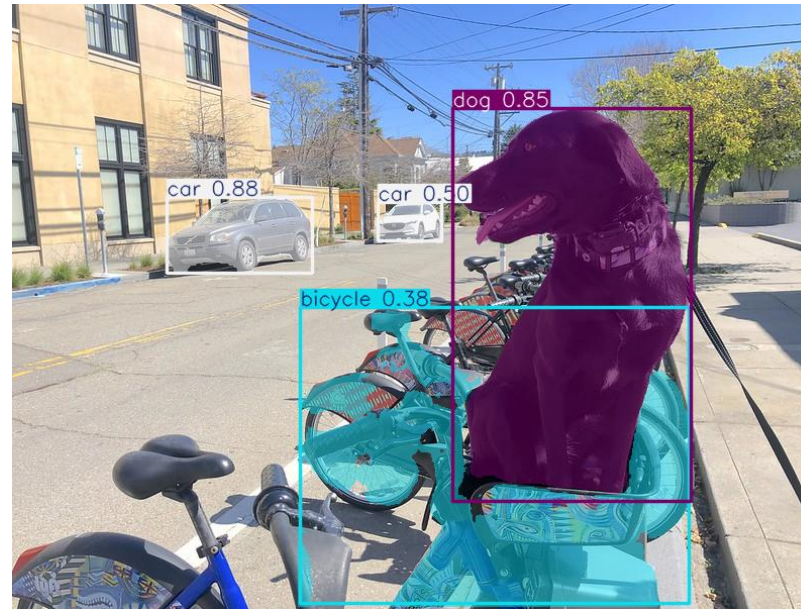
https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-instance-segmentation.ipynb

Yolo26 不同模型實例分割效果比較

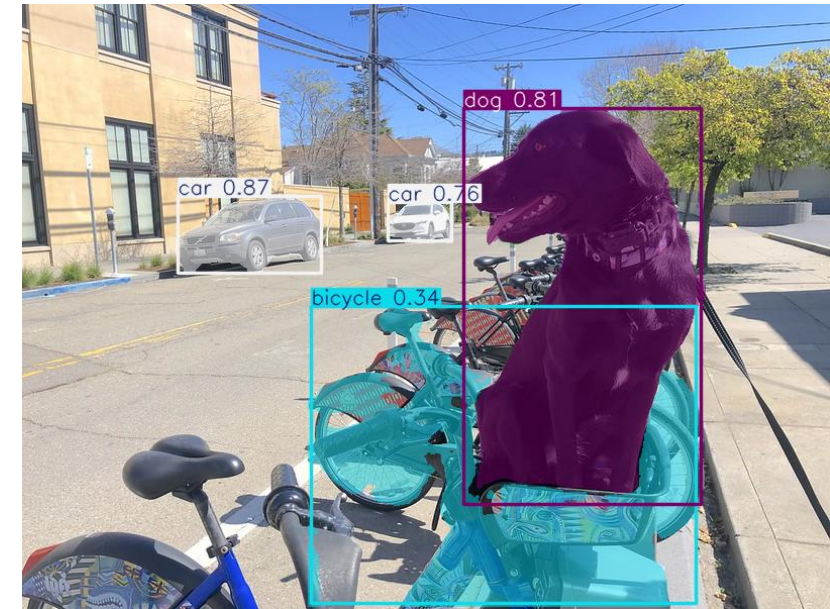
YOLO26 PyTorch



YOLO26 OpenVINO IR



YOLO26 OpenVINO NNCF



https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-instance-segmentation.ipynb

AI 影像分割技術 — 基於Transformer

Meta - **S**egment **A**nanything **M**odel, **SAM**

ViT影像編碼器、提示編碼器、輕量遮罩解碼器
衍生輕量技術：**FastSAM, MobileSAM, EfficientSAM**

SAM

- 2023/04發佈，基於SA-1B資料集，只適用靜態影像。
- 互動式（點擊、方框、文字指示）即時生成遮罩。

（SA-1B，包含約 1,100 萬張高解析度影像及超過 11 億個高質量分割遮罩）

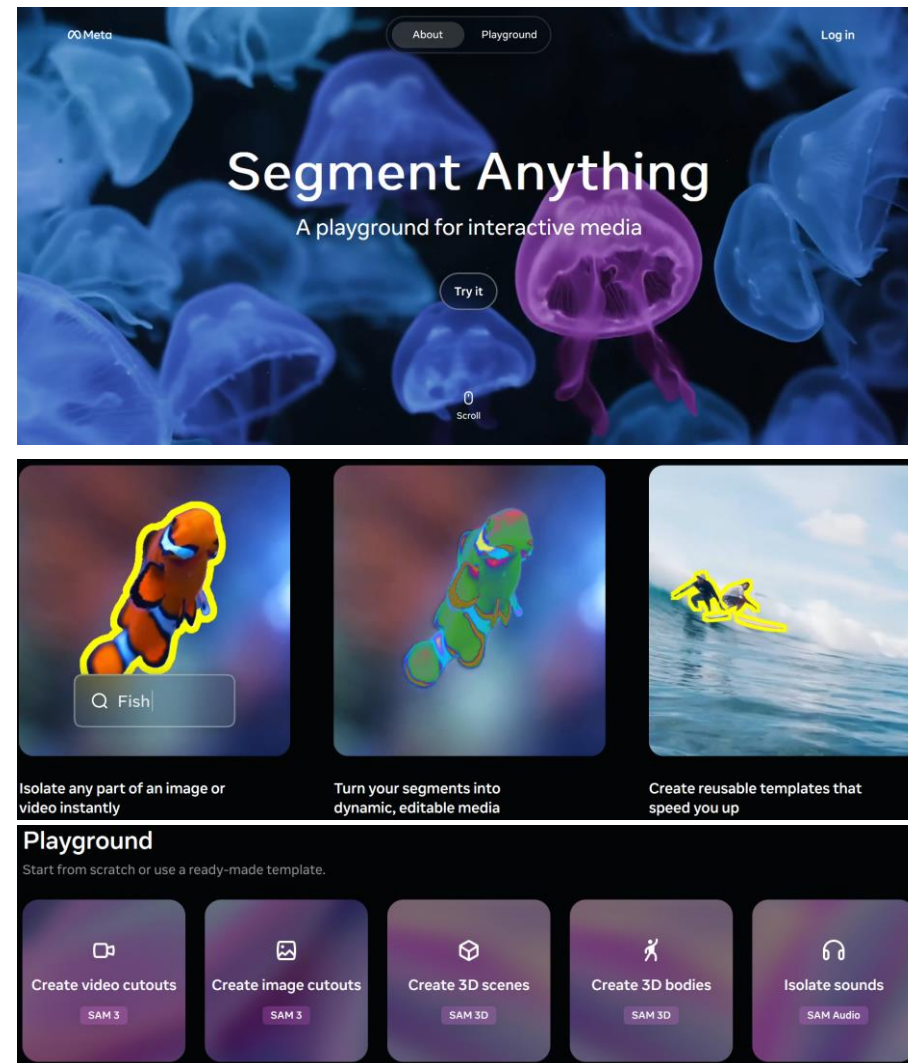
SAM2

- 2024/07發佈，將能力延伸至動態影片，包括追蹤。
- 可處理影片中物件短暫遮蔽，處理效能提升六倍。

SAM3

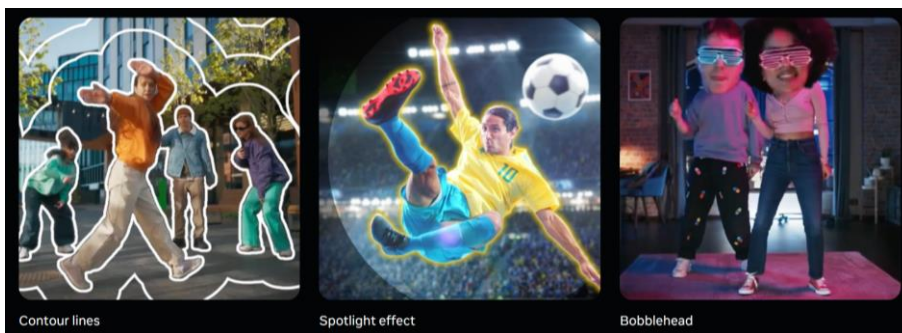
- 2025/11發佈，採用語義理解及多模態融合。
- 可進行複雜邏輯推理，SAM3D還可從2D提取3D點雲。

<https://aidemos.meta.com/segment-anything/>

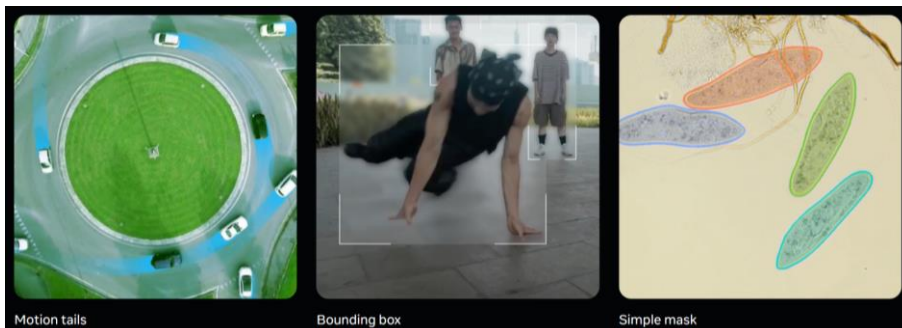


Meta SAM 影像分割主要應用

特色
模版



追蹤
物件



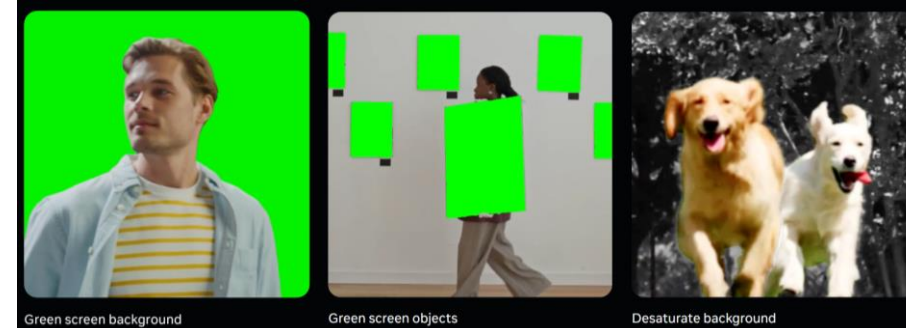
就是
有趣



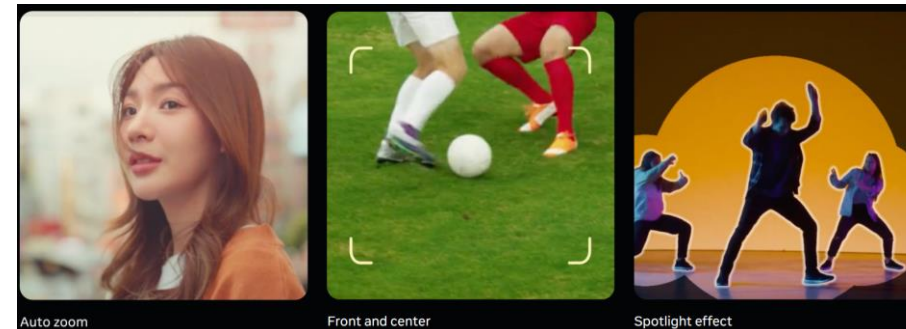
隱藏
物件



每日
編輯



創建
故事



<https://aidemos.meta.com/segment-anything/gallery>

Intel OpenVINO SAM 相關範例

SAM, SAM2, SAM3, FastSAM, EfficientSAM, GroundedSAM (GroundingDINO + SAM)



AI Trends SAM 3 Image Segmentation with OpenVINO

Model Demos • Image Segmentation

[View on GitHub](#) [Show Status](#) [Link](#)



AI Trends Object masks from prompts with SAM2 and OpenVINO

Model Demos • Image Segmentation

[View on GitHub](#) [Show Status](#) [Link](#)



AI Trends Object masks from prompts with SAM2 and OpenVINO

Model Demos • Image Segmentation

[View on GitHub](#) [Show Status](#) [Link](#)



AI Trends Object detection and masking from prompts with GroundingDINO and OpenVINO

Convert • Optimize • Model Demos • Live Demos • Object Detection • Image Segmentation

[View on GitHub](#) [Open in Colab](#) [Show Status](#) [Link](#)

Text Prompt: "Horse, Clouds, Grasses, Sky, Hill."



AI Trends Object segmentations with EfficientSAM and OpenVINO

Model Demos • Image Segmentation

[View on GitHub](#) [Show Status](#) [Link](#)



AI Trends Object segmentations with FastSAM and OpenVINO

Model Demos • Image Segmentation

[View on GitHub](#) [Open in Colab](#) [Launch in Binder](#) [Show Status](#) [Link](#)



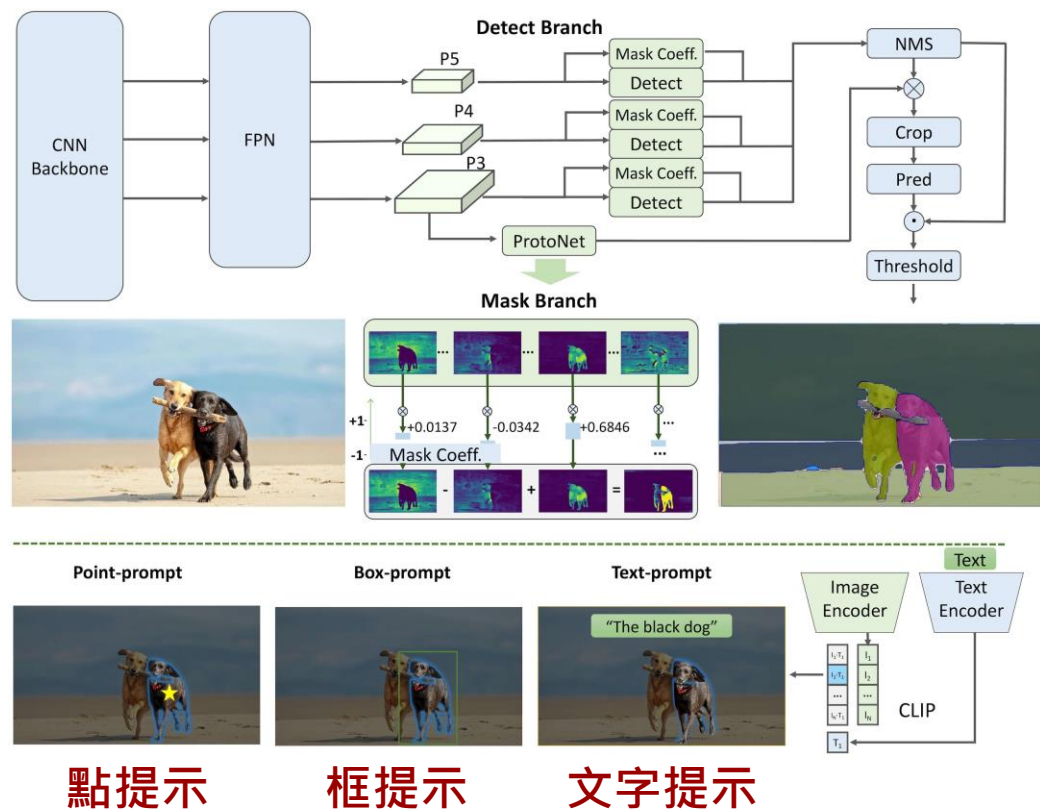
AI Trends Object masks from prompts with SAM and OpenVINO

Model Demos • Image Segmentation

[View on GitHub](#) [Show Status](#) [Link](#)

https://openvinotoolkit.github.io/openvino_notebooks/?search=SAM

範例7-2：OpenVINO Fast SAM 實例分割



- [Prerequisites](#)
 - [Install requirements](#)
 - [Imports](#)
- [FastSAM in Ultralytics](#)
- [Convert the model to OpenVINO Intermediate representation \(IR\) format](#)
- [Embedding the converted models into the original pipeline](#)
 - [Select inference device](#)
 - [Adapt OpenVINO models to the original pipeline](#)
- [Optimize the model using NNCF Post-training Quantization API](#) (約75~80min, 可略過)
 - [Compare the performance of the Original and Quantized Models](#) (約55~60min, 可略過)
- [Try out the converted pipeline](#)



Model Format	Preprocess	Inference	Postprocess	Output
PyTorch	16.7ms	12,666ms	172.8ms	768x1024,37 object
IR	17.3ms	18,420ms	170.6ms	1024x1024,42 object

說明文檔：https://github.com/openvinotoolkit/openvino_notebooks/tree/latest/notebooks/fast-segment-anything
 範例程式：https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/fast-segment-anything/fast-segment-anything.ipynb

範例7-2：Gradio 人機介面

Input



拖放圖片至此處
- 或 -
點擊上傳





Segmentation Map



Pixel selector type

☒ Object point
 ☐ Background point
 ☐ Bounding Box

Select model variant

☒ FP32 model
 ☐ Quantized model

Segment

Clear points

Examples





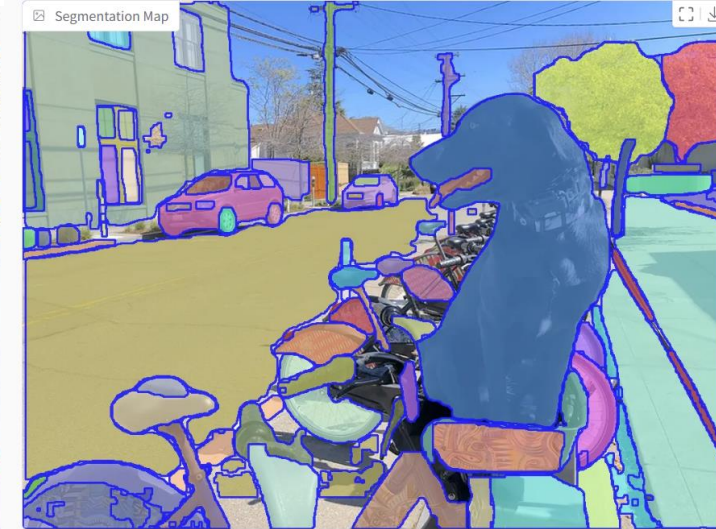
範例7-2：Fast SAM 實例分割結果



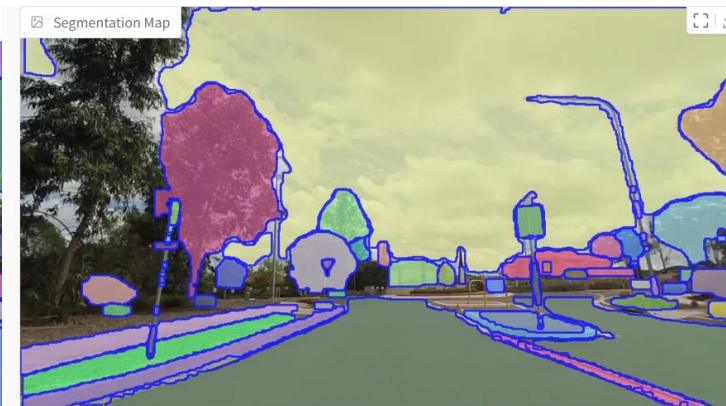
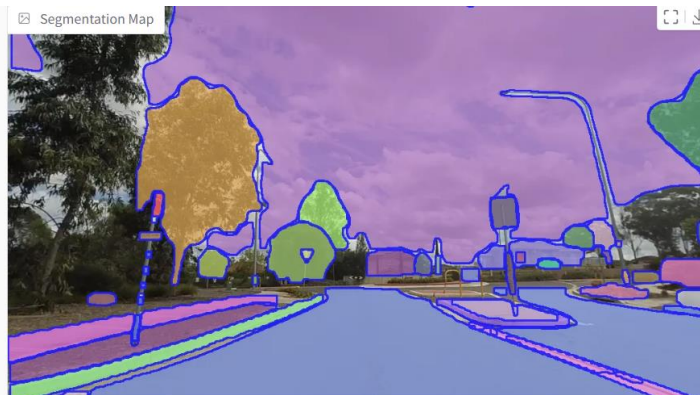
原始影像



FP32 Model



Quantized Model



使用 Gradio 設計圖形化人機介面



如何使用 Gradio 快速搭建
人工智慧應用圖形化人機介面

說明文檔：<https://omnixri.blogspot.com/2024/12/vmaker-edge-ai-24-gradio.html>

範例程式：https://colab.research.google.com/github/OmniXRI/Hello_World_Gradio/blob/main/Hello_World_Gradio.ipynb

範例7-3 : Ultralytics SAM (1/2)

```
from ultralytics import SAM

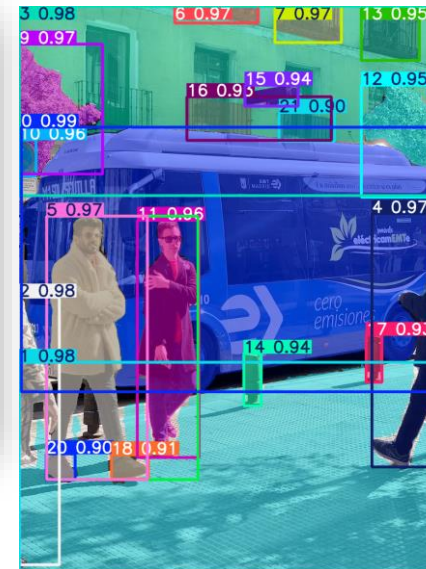
# Load a model
# For SAM=sam_b.pt, SAM 2=sam2_b.pt, SAM 2.1=sam2.1_b.pt (default, runs out-of-the-box)
# Upgrade to SAM 3 by setting model="sam3.pt". SAM 3 weights require approval + download from
# https://huggingface.co/facebook/sam3 — place sam3.pt in your working directory first.
model = SAM("sam2.1_b.pt")

model.info() # Display model information (optional)

# Run inference (image or video)
results = model("https://ultralytics.com/images/bus.jpg") # image
# results = model("https://youtu.be/LNwODJXcvt4") # video file

results[0].show() # Display results
```

Segment Everything



```
from ultralytics import SAM

# Load a model (swap to "sam3.pt" if you have SAM 3 access)
model = SAM("sam2.1_b.pt")

# Run inference with bboxes prompt (Provide the bounding box coordinates
# for the bus area, ensuring that only bus is segmented in the entire image)
results = model("https://ultralytics.com/images/bus.jpg",
                bboxes=[3.8328723907470703, 229.35601806640625,
                        796.2098999023438, 728.4313354492188])

results[0].show() # Display results
```

Bounding box prompt

本範例可支援推論模型
sam_b, sam2_b,
sam2.1_b, sam2_t,
sam3_b, mobile_sam

說明文檔 : <https://docs.ultralytics.com/zh/models/sam/>

範例程式 : <https://colab.research.google.com/github/ultralytics/notebooks/blob/main/notebooks/inference-with-meta-sam-and-sam2-using-ultralytics-python-package.ipynb>

範例7-3 : Ultralytics SAM (2/2)

```
from ultralytics import SAM

# Load a model (swap to "sam3.pt" if you have SAM 3 access)
model = SAM("sam2.1_b.pt")

# Run inference with point prompt (Provide the point coordinates for the
# person area, ensuring that only the person is segmented in the entire image)
results = model("https://ultralytics.com/images/bus.jpg",
                points=[34, 714])

results[0].show() # Display results
```

```
from ultralytics import SAM

# Load a model (swap to "sam3.pt" if you have SAM 3 access)
model = SAM("sam2.1_b.pt")

# Run inference with multiple point prompts (Provide the points coordinates for
# person area, ensuring that only the person is segmented in the entire image)
results = model("https://ultralytics.com/images/bus.jpg",
                points=[[34, 714], [283, 634]])

results[0].show() # Display results
```



說明文檔 : <https://docs.ultralytics.com/zh/models/sam/>

範例程式 : <https://colab.research.google.com/github/ultralytics/notebooks/blob/main/notebooks/inference-with-meta-sam-and-sam2-using-ultralytics-python-package.ipynb>

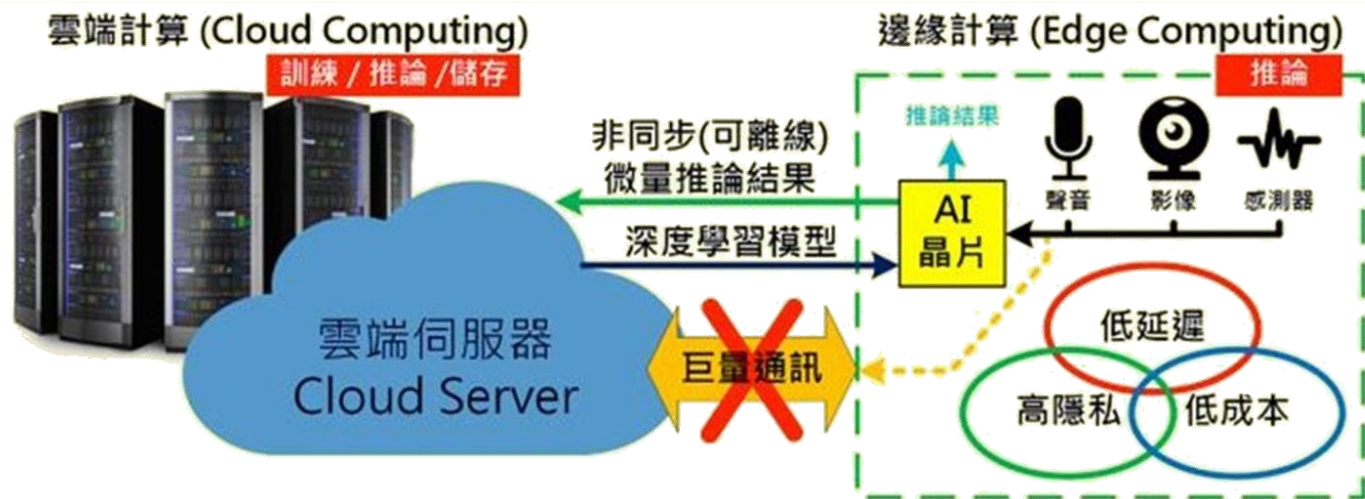
SAM 系列 vs. YOLO-seg 性能比較

模型	大小 (MB)	参数 (M)	速度 (CPU) (ms/im)
Meta SAM-b	375	93.7	41703
Meta SAM2-b	162	80.8	28867
Meta SAM2-t	78.1	38.9	23430
MobileSAM	40.7	10.1	23802
帶有 YOLOv8 backbone 的 FastSAM-s	23.9	11.8	58.0
Ultralytics YOLOv8n-seg	7.1 (缩小至原来的1/11)	3.4 (低 11.4 倍)	24.8 (速度提升945倍)
Ultralytics YOLO11n-seg	6.2 (缩小至原来的1/12.6)	2.9 (低 13.4 倍)	24.3 (速度提升964倍)
Ultralytics (Yolo26)	6.7 (小11.7倍)	2.7 (减少14.4倍)	25.2 (速度提升930倍)

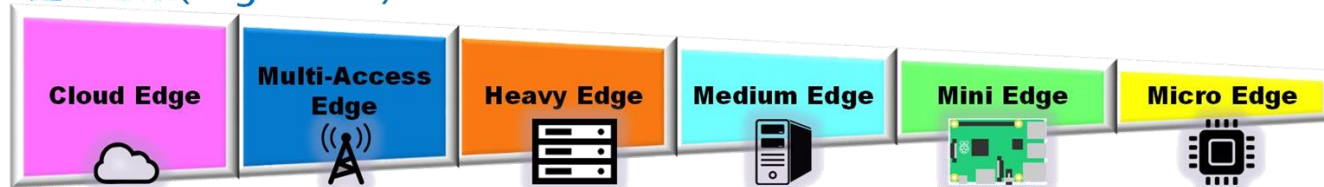
資料來源：<https://docs.ultralytics.com/zh/models/mobile-sam/#mobilesam-comparison-vs-yolo>



Edge AI



邊緣等級(Edge Level)



7.2. 姿態估測

姿態估測(Pose Estimation)簡介

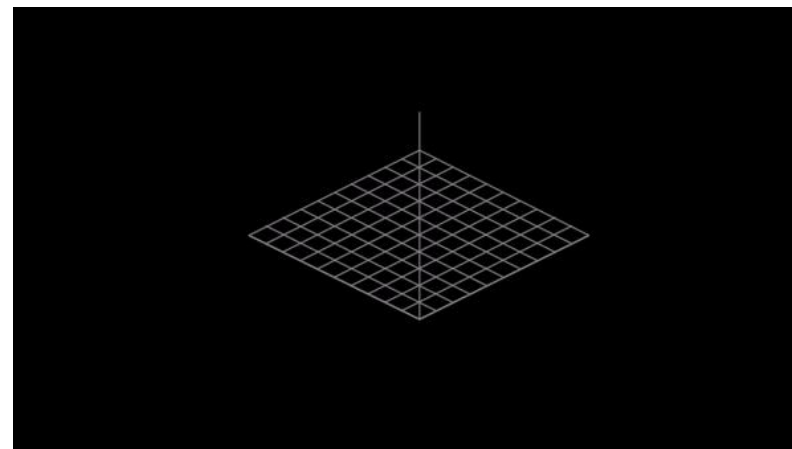
主要部位

- 頭部
- 注視點
- 手部
- 全身



估測方式

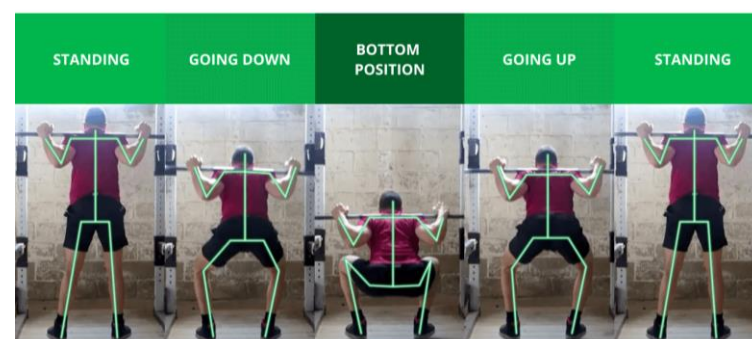
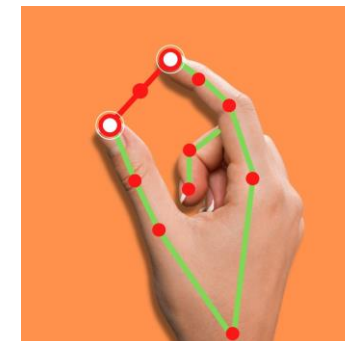
- 2D (照片、攝影機)
- 3D (深度攝影機)
- 2.5D (假3D，輸入2D，輸出3D，以平面估測立體深度)



姿態估測常見用途

常見用途

- 人機互動
- 運動分析
- 組裝確認
- 智慧零售
- 智慧安防
- 智慧教室
- 安全駕駛
- ...



姿態估測（人體關鍵點）資料集

常見的2D關鍵點開放資料集

- 微軟COCO (17點)
- CMU OpenPose (18/25點)
 - 人體骨架 (18/25點)
 - 臉部動作 (70點)
 - 手部動作 (22點)
- MPII (16點)
- AI Challenge (14點)
- LSP (14點)
- FLIC (9點)



<https://cocodataset.org/#keypoints-2020>

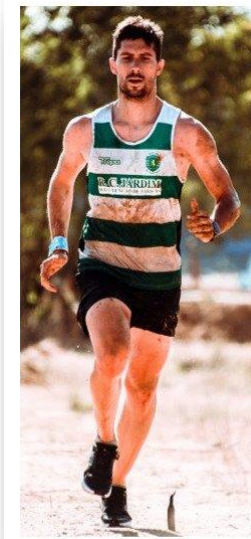
Microsoft COCO 關鍵點資料集



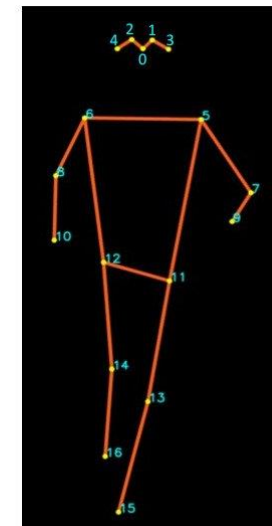
Person_keypoints_val2017 範例 (JSON)

```
annotation{
  "keypoints": [x1,y1,v1,...],
  "num_keypoints": int,
  "id": int,
  "image_id": int,
  "category_id": int,
  "segmentation": RLE or [polygon],
  "area": float,
  "bbox": [x,y,width,height],
  "iscrowd": 0 or 1,
}
```

v=0 未標註, xy須為0
v=1 已標註但不可見
v=2 標註且可見



Index	Key point
0	Nose
1	Left-eye
2	Right-eye
3	Left-ear
4	Right-ear
5	Left-shoulder
6	Right-shoulder
7	Left-elbow
8	Right-elbow
9	Left-wrist
10	Right-wrist
11	Left-hip
12	Right-hip
13	Left-knee
14	Right-knee
15	Left-ankle
16	Right-ankle



Categories字串

```
{
  "id": int,
  "name": str,
  "supercategory": str,
  "keypoints": [str],
  "skeleton": [edge]
}
```

關鍵點名稱 (17)
骨架連結

資料來源：<https://cocodataset.org/#keypoints-2020>, <https://cocodataset.org/#keypoints-eval>

CMU OpenPose 基本介紹

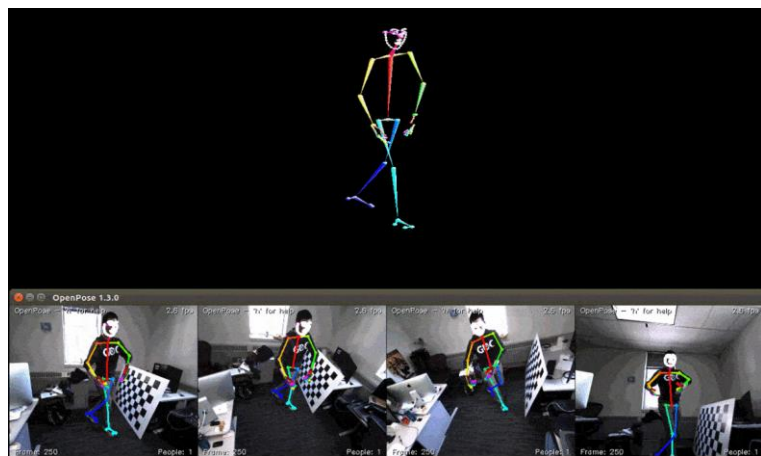


➤ 卡內基美濃大學(CMU)
2017年提出。

➤ 開源人體姿態專案，包含即時人體動作(2D/3D)、面部表情及手指運動偵測等。

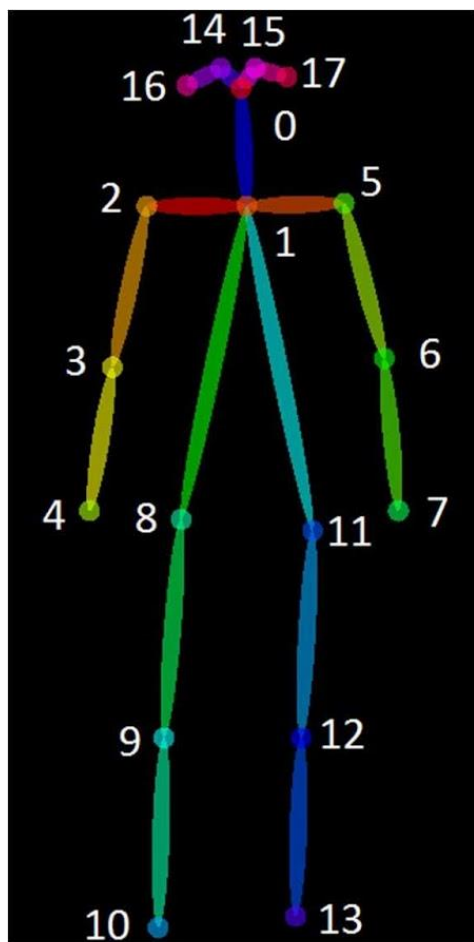
➤ 支援Windows, Mac OS, Linux(Ubuntu)。

➤ 支援CPU及GPU(CUDA, OpenCL)加速。

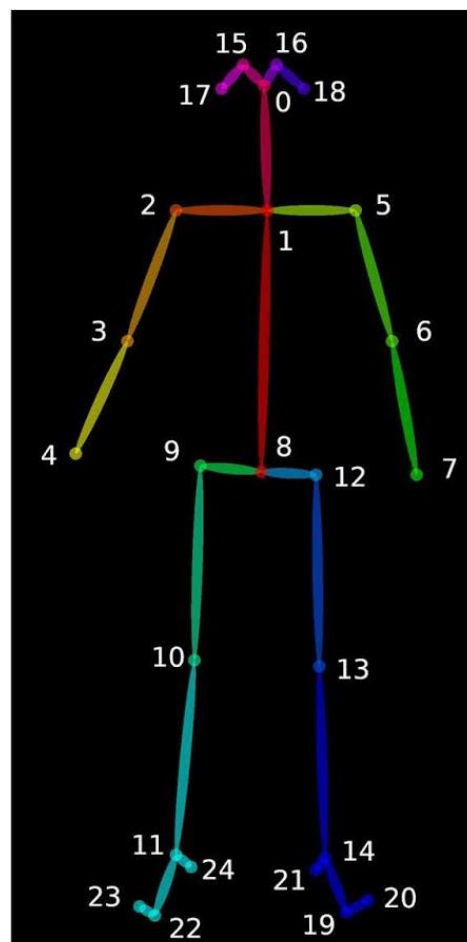


<https://github.com/CMU-Perceptual-Computing-Lab/openpose>

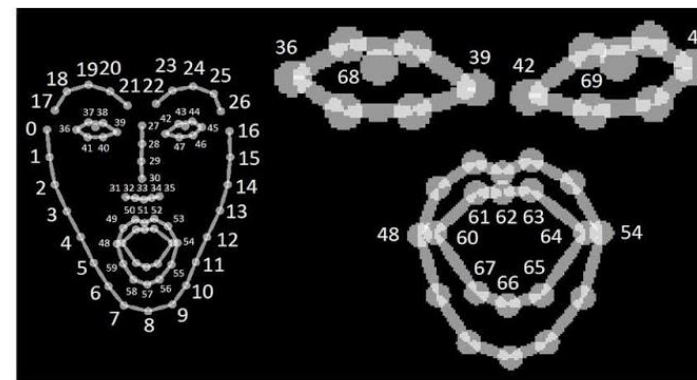
OpenPose 關鍵點定義



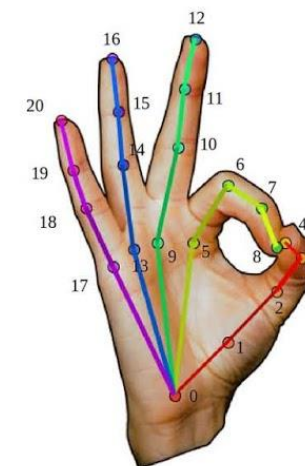
(a)



(b)



(b)



(d)

OpenPose支援輸出格式，(a)類COCO 18點，(b) BODY 25點，(c) 臉部70點，(d)手部22點。

姿態關鍵點轉換

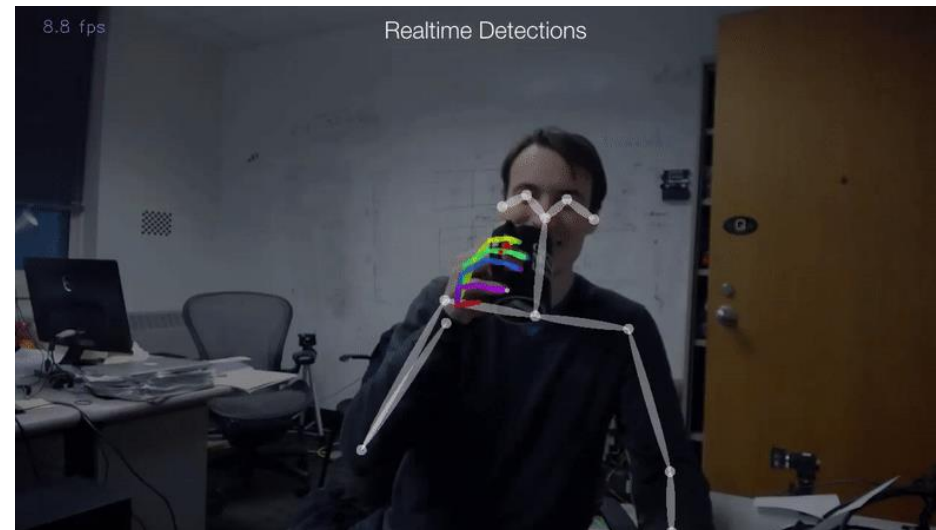
MS COCO: (17點)

{0:鼻子, 1:左眼, 2:右眼, 3:左耳, 4:右耳, 5:左肩, 6:右肩, 7:左肘, 8:右肘, 9:左腕, 10:右腕, 11:左臀 (腰), 12:右臀 (腰), 13:左膝, 14:右膝, 15:左踝, 16:右踝 }

CMU OpenPose: (18點)

{0:鼻子, 1:脖子, 2:右肩, 3:右肘, 4:右腕, 5:左肩, 6:左肘, 7:左腕, 8:右臀 (腰), 9:右膝, 10:右踝, 11:左臀 (腰), 12:左膝, 13:左踝, 14:左眼, 15:右眼, 16:左耳, 17:右耳 }

OpenPose 結果展示



Intel OpenVINO 姿態估測範例



AI Trends Convert and Optimize YOLO26 keypoint detection model with OpenVINO™

Model Demos • Pose Estimation

[View on GitHub](#) [Open in Colab](#) [Show Status](#) [Link](#)



AI Trends Convert and Optimize YOLOv11 keypoint detection model with OpenVINO™

Model Demos • Pose Estimation

[View on GitHub](#) [Open in Colab](#) [Show Status](#) [Link](#)



AI Trends Convert and Optimize YOLOv8 keypoint detection model with OpenVINO™

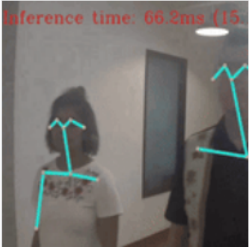
Model Demos • Pose Estimation

[View on GitHub](#) [Open in Colab](#) [Show Status](#) [Link](#)

YOLO v8 / v11 / 26 Keypoint

https://openvinotoolkit.github.io/openvino_notebooks/?search=YOLO

2D / 3D Pose Estimation



Inference time: 66.2ms (15)

Live 3D Human Pose Estimation with OpenVINO

Live Demos • Pose Estimation

[View on GitHub](#) [Launch in Binder](#) [Show Status](#) [Link](#)



Live Human Pose Estimation with OpenVINO™

Live Demos • Pose Estimation

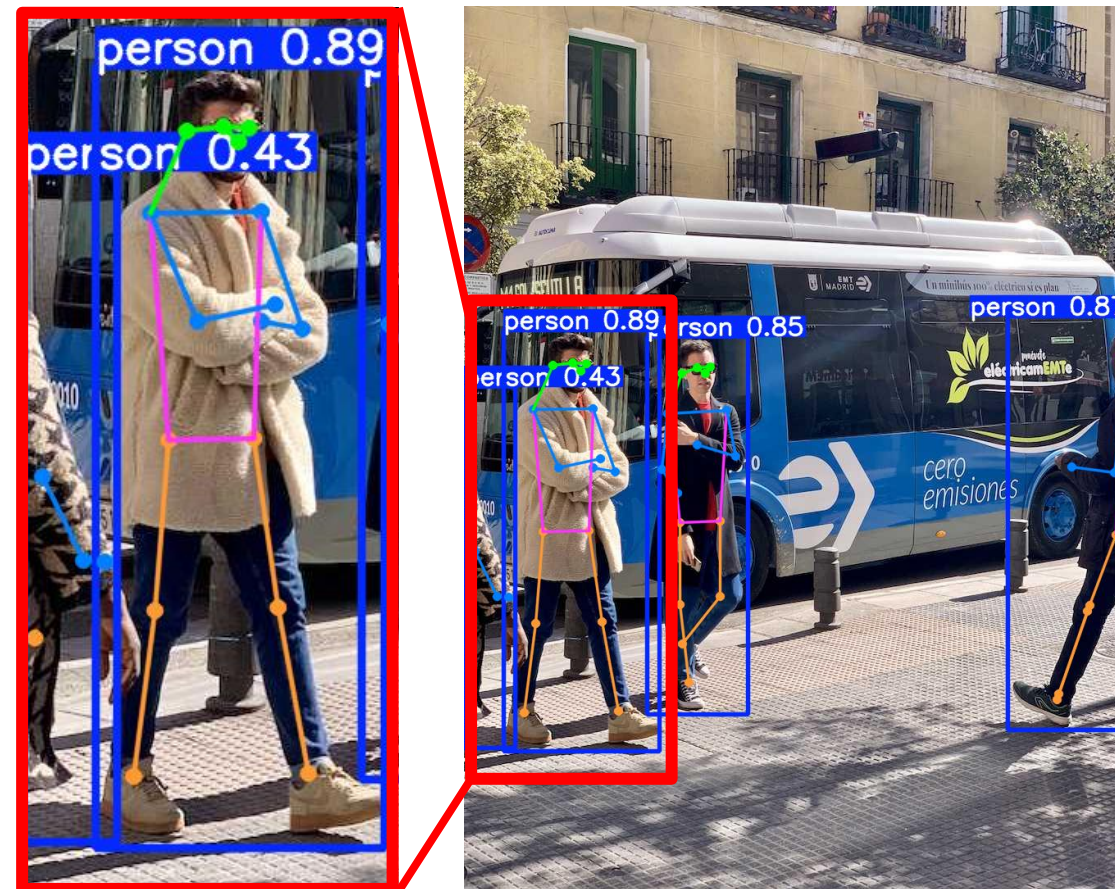
[View on GitHub](#) [Launch in Binder](#) [Show Status](#) [Link](#)

https://openvinotoolkit.github.io/openvino_notebooks/?search=pose

範例7-4：OpenVINO YOLO26 姿態估測

- [Get PyTorch model](#)
 - [Prerequisites](#)
- [Instantiate model](#)
 - [Convert model to OpenVINO IR](#)
 - [Verify model inference](#)
 - [Select inference device](#)
 - [Test on single image](#)
- [Optimize model using NNCF Post-training Quantization API](#)
 - [Validate Quantized model inference](#)
- [Compare performance of the Original and Quantized Models](#)
- [Other ways to optimize model](#)
- [Live demo](#)
 - [Run Keypoint Detection on video](#)

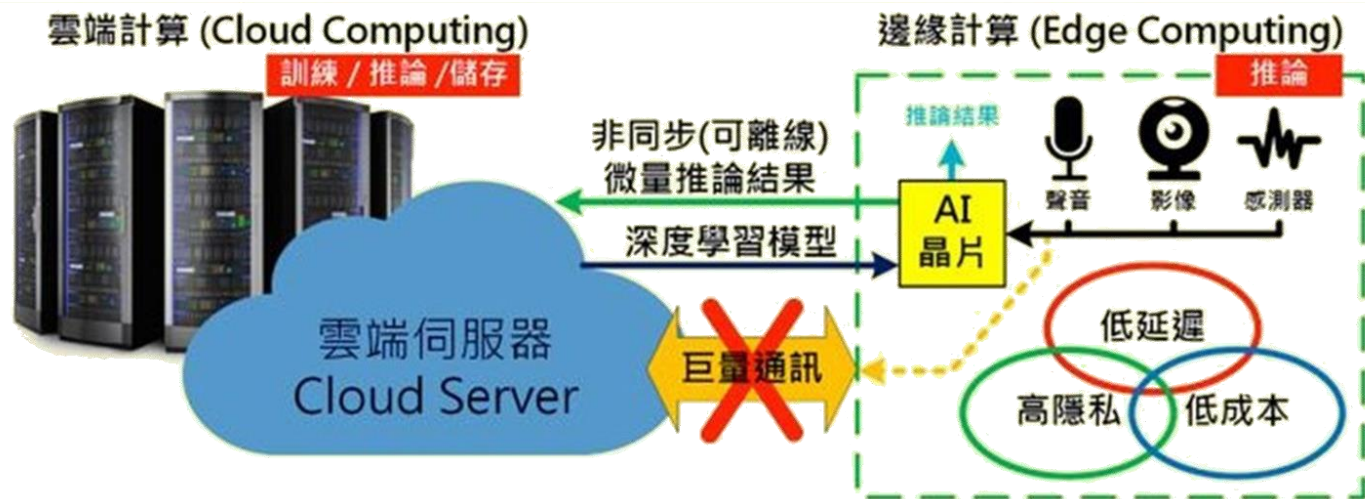
可支援推論模型
yolo26n-pose
yolo26s-pose
yolo26m-pose
yolo26l-pose
yolo26x-pose



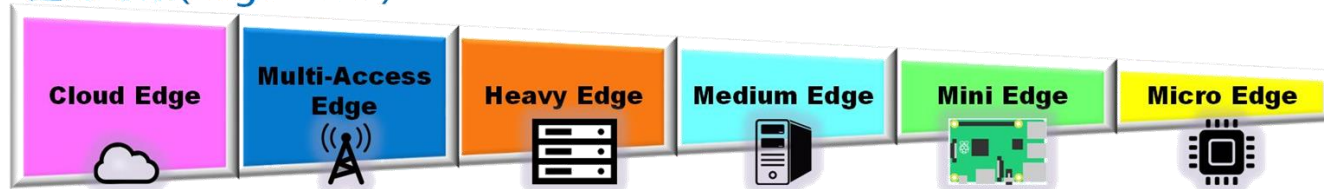
說明文檔：https://github.com/openvinotoolkit/openvino_notebooks/tree/latest/notebooks/yolov26-optimization
 範例程式：https://colab.research.google.com/github/openvinotoolkit/openvino_notebooks/blob/latest/notebooks/yolov26-optimization/yolov26-keypoint-detection.ipynb



Edge AI

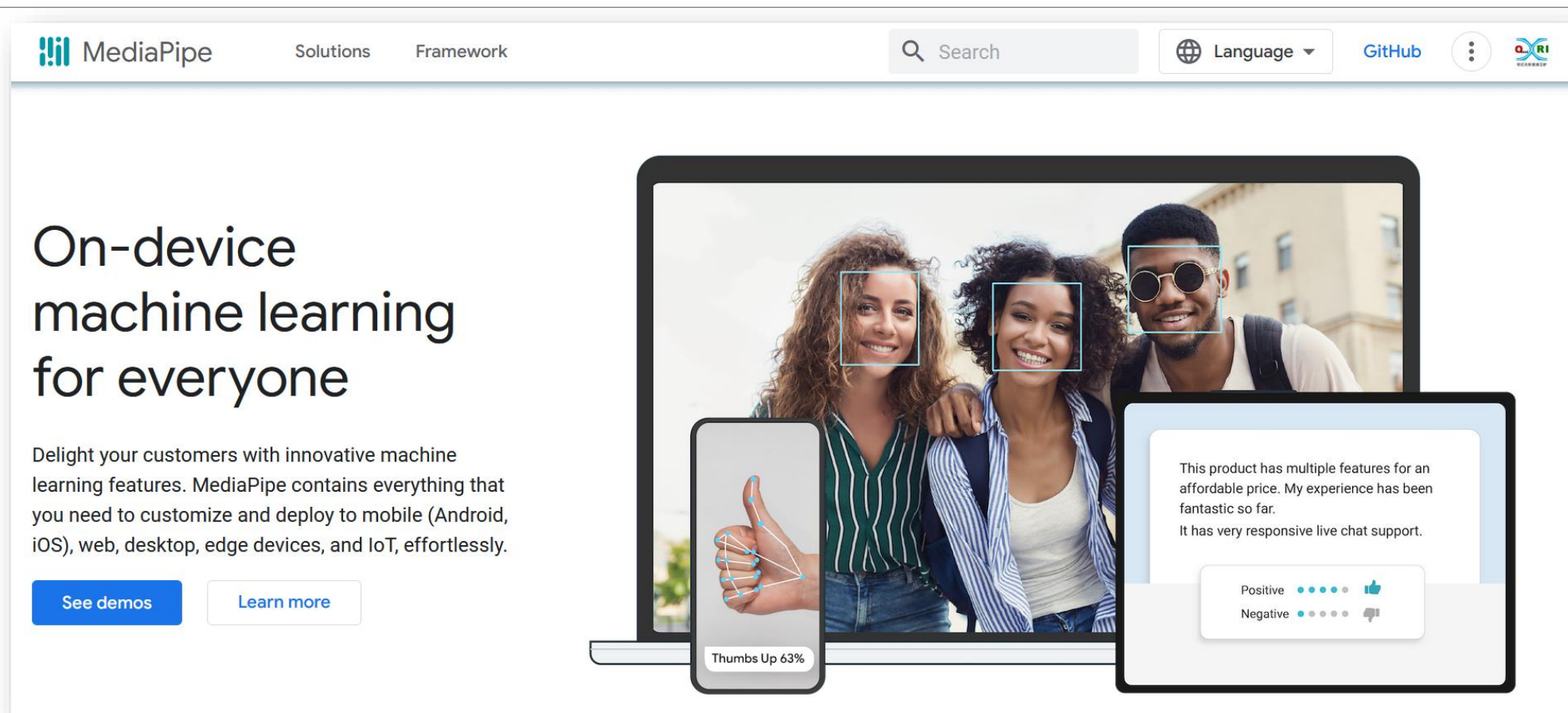


邊緣等級(Edge Level)



7.3. GOOGLE MEDIAPIPE

Google AI Edge – MediaPipe簡介



2019/6 首次於**CVPR**
展示預覽版本

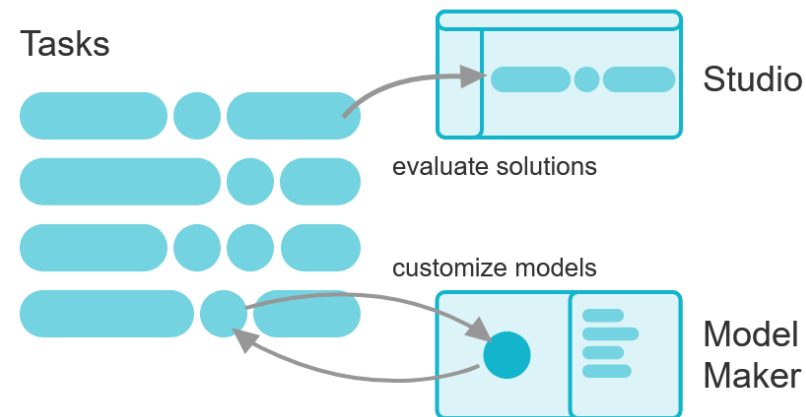
開發文件：<https://developers.google.com/mediapipe>
Github：<https://github.com/google-ai-edge/mediapipe>

Google MediaPipe 可用解決方案

解決方案	Android	網頁版	Python	iOS	自訂模型
LLM Inference API	●	●		●	●
物件偵測	●	●	●	●	●
圖片分類	●	●	●	●	●
圖片分割	●	●	●		
互動式區隔	●	●	●		
手部地標偵測	●	●	●	●	
手勢辨識	●	●	●	●	●
圖片嵌入	●	●	●		
臉部偵測	●	●	●	●	
臉部特徵偵測	●	●	●		
姿勢地標偵測	●	●	●		
圖像生成	●				●
文字分類	●	●	●	●	●
文字嵌入	●	●	●		
語言偵測器	●	●	●		
音訊分類	●	●	●		

自 2023 年 3 月 1 日起，部份舊版 MediaPipe 解決方案已停用或升級，可參考 <https://developers.google.com/mediapipe>。

輕量、跨平台、跨硬體



- **MediaPipe Tasks**：用於部署解決方案的跨平台 API 和程式庫。
- **MediaPipe 模型**：經過預先訓練且立即可用的模型，適用於各項解決方案。
- **MediaPipe Model Maker**：使用資料自訂解決方案模型。瞭解詳情
- **MediaPipe Studio**：在瀏覽器中以視覺化方式呈現、評估及基準化解決方案。

Google MediaPipe 解決方案展示



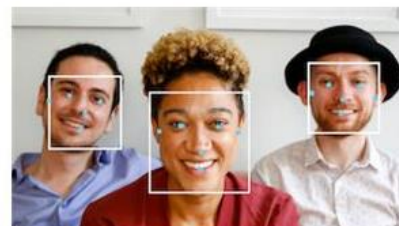
影像辨識



物件偵測



手勢辨識



人臉偵測



影像相似度



影像分割



交互式分割



手部特徵點



臉部特徵點



姿態特徵點



文字情緒



文字相似度



語系辨識



語音辨識



DEMO: <https://mediapipe-studio.webapps.google.com/home>

Google MediaPipe Tasks WEB測試頁

Vision

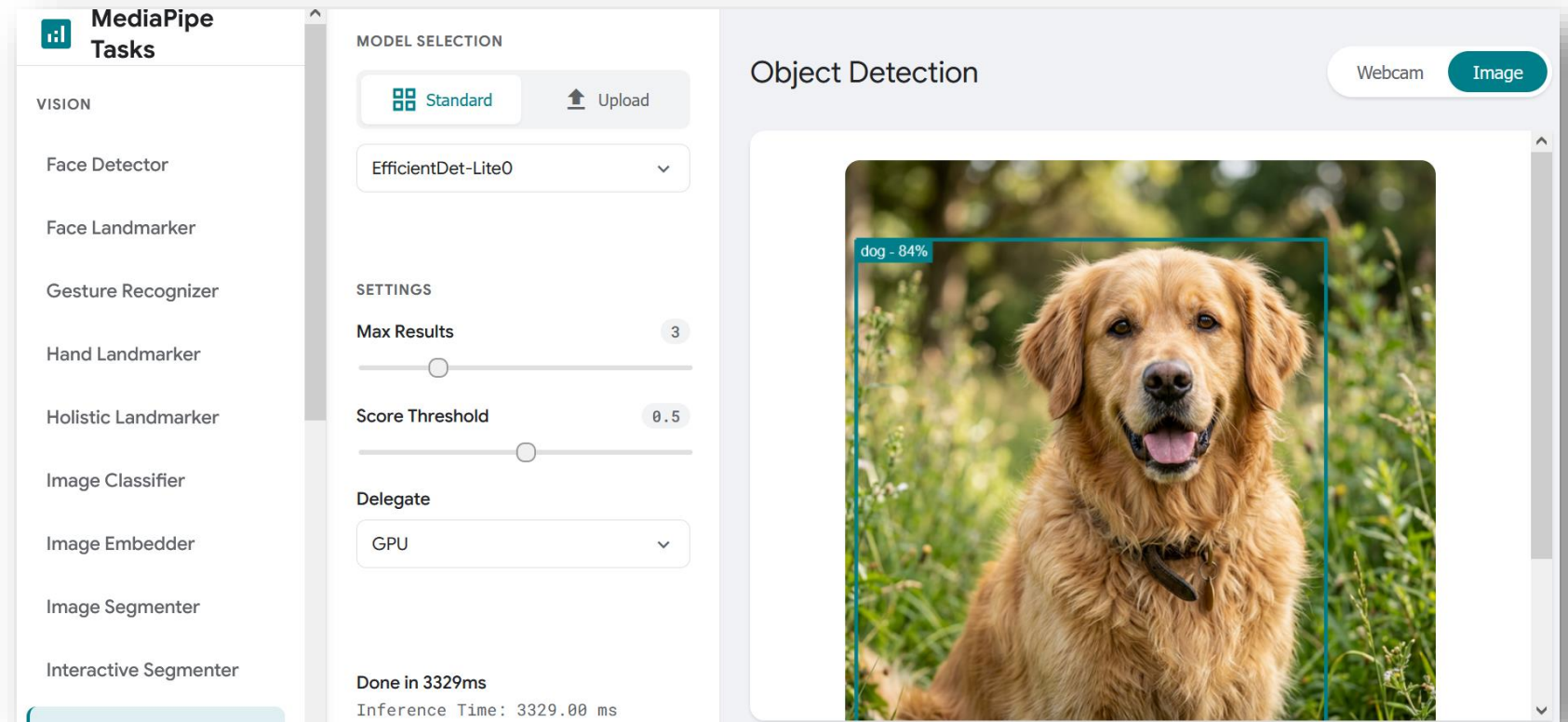
Face Detector
Face Landmarker
Gesture Recognizer
Hand Landmarker
Holistic Landmarker
Image Classifier
Image Embedder
Image Segmenter
Interactive Segmenter
Object Detector
Pose Landmarker

Audio

Audio Classifier

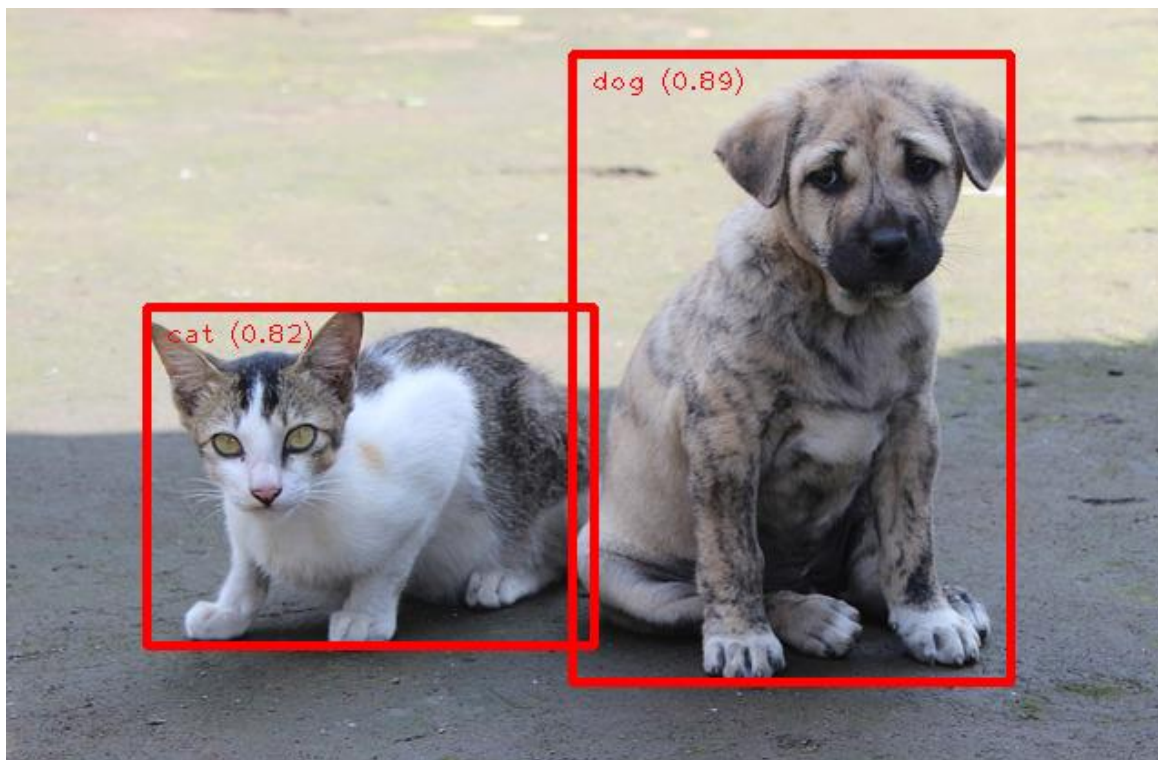
Text

Language Detector
Text Classifier
Text Embedder



<https://google-ai-edge.github.io/mediapipe-samples-web/>

物件偵測 (Object Detection)



COCO Dataset – 80 classes

EfficientDet-Lite0 (int8)	320 x 320	int8
EfficientDet-Lite0 (float 16)	320 x 320	float 16
EfficientDet-Lite0 (float 32)	320 x 320	None (float32)
EfficientDet-Lite2 (int8)	448 x 448	int8
EfficientDet-Lite2 (浮點 16)	448 x 448	float 16
EfficientDet-Lite2 (浮點 32)	448 x 448	None (float32)
SSDMobileNet-V2 (int8)	256 x 256	int8
SSDMobileNet-V2 (浮點 32)	256 x 256	None (float32)

說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/object_detector?hl=zh-tw

範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/object_detection/python/object_detector.ipynb

影像分類 (Image Classification)

cheeseburger (0.98)



ImageNet Dataset – 1000 classes

EfficientDet-Lite0 (int8)	320 x 320	int8
EfficientDet-Lite0 (float 16)	320 x 320	float 16
EfficientDet-Lite0 (float 32)	320 x 320	None (float32)
EfficientDet-Lite2 (int8)	448 x 448	int8
EfficientDet-Lite2 (浮點 16)	448 x 448	float 16
EfficientDet-Lite2 (浮點 32)	448 x 448	None (float32)

說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/object_detector?hl=zh-tw

範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/image_classification/python/image_classifier.ipynb

影像分割 (Image Segmentation)

原始影像



影像分割、產生遮罩



背景模糊化



自拍分割

SelfieSegmenter (正方形)	256 x 256	float 16
SelfieSegmenter (橫向)	144 x 256	float 16

頭髮分割

0 - background
1 - hair

HairSegmenter	512 x 512	None (float32)
---------------	-----------	----------------

多元自拍分割

0 - background 3 - face-skin
1 - hair 4 - clothes
2 - body-skin 5 - others (accessories)

SelfieMulticlass (256 x 256)	256 x 256	None (float32)
------------------------------	-----------	----------------

DeepLab-v3
COCO Dataset – 20 classes (相容VOC)

DeepLab-V3	257 x 257	None (float32)
------------	-----------	----------------

說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/image_segmenter?hl=zh-tw

範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/image_segmentation/python/image_segmentation.ipynb

互動式分割 (Interactive_segmentation)

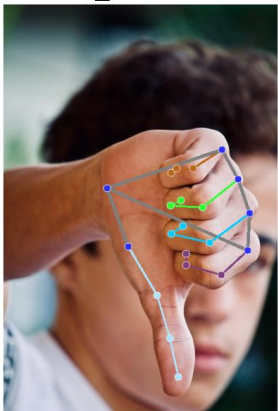


說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/interactive_segmenter?hl=zh-tw

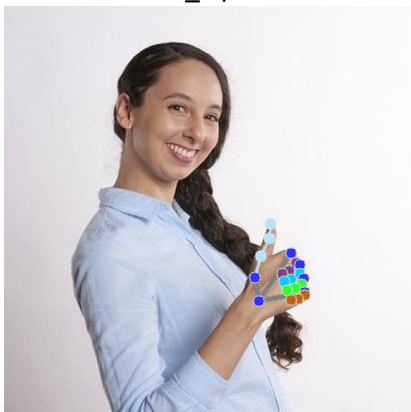
範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/interactive_segmentation/python/interactive_segmenter.ipynb

手勢辨識 (Gesture Recognition)

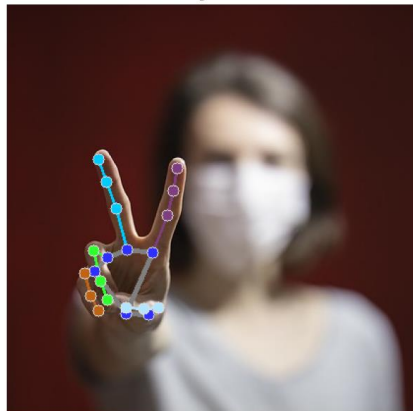
Thumb_Down (0.77)



Thumb_Up (0.73)



Victory (0.91)



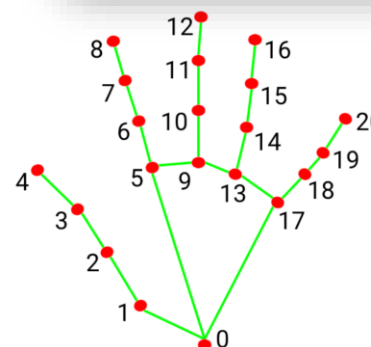
Pointing_Up (0.82)



HandGestureClassifier

192 x 192 、 224 x 224

浮點數 16



- | | |
|-----------------------|-----------------------|
| 0. WRIST | 11. MIDDLE_FINGER_DIP |
| 1. THUMB_CMC | 12. MIDDLE_FINGER_TIP |
| 2. THUMB_MCP | 13. RING_FINGER_MCP |
| 3. THUMB_IP | 14. RING_FINGER_PIP |
| 4. THUMB_TIP | 15. RING_FINGER_DIP |
| 5. INDEX_FINGER_MCP | 16. RING_FINGER_TIP |
| 6. INDEX_FINGER_PIP | 17. PINKY_MCP |
| 7. INDEX_FINGER_DIP | 18. PINKY_PIP |
| 8. INDEX_FINGER_TIP | 19. PINKY_DIP |
| 9. MIDDLE_FINGER_MCP | 20. PINKY_TIP |
| 10. MIDDLE_FINGER_PIP | |

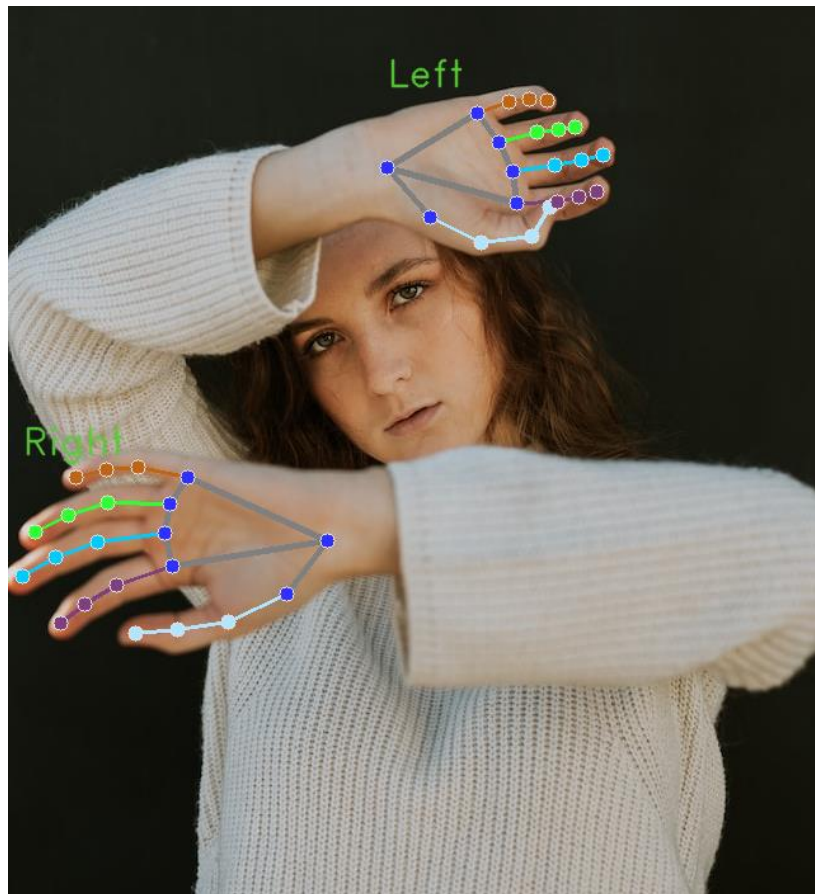
- 0 - Unrecognized gesture, label: Unknown
- 1 - Closed fist, label: Closed_Fist
- 2 - Open palm, label: Open_Palm
- 3 - Pointing up, label: Pointing_Up
- 4 - Thumbs down, label: Thumb_Down
- 5 - Thumbs up, label: Thumb_Up
- 6 - Victory, label: Victory
- 7 - Love, label: ILoveYou



說明文檔 : https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer?hl=zh-tw

範例程式 : https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/gesture_recognizer/python/gesture_recognizer.ipynb

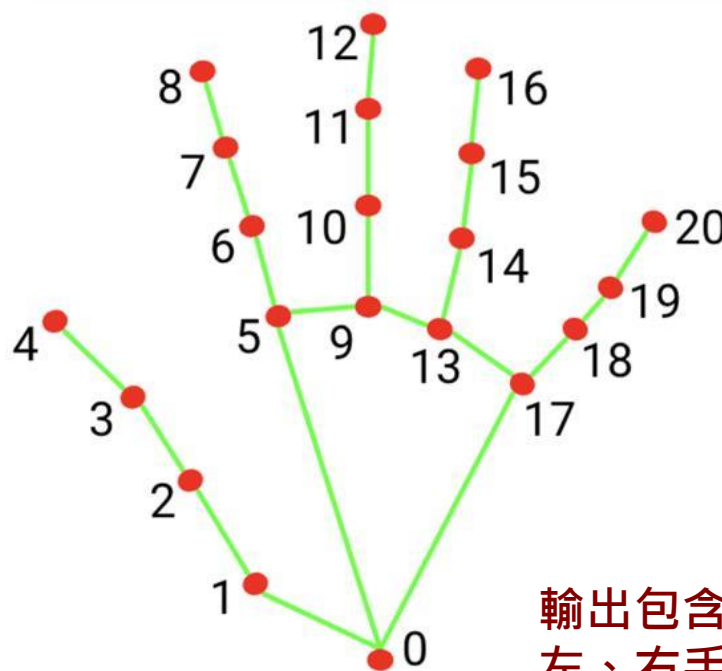
手部特徵點偵測 (Hand landmark detection)



HandLandmarker (完整)

192 x 192 、 224 x 224

float 16



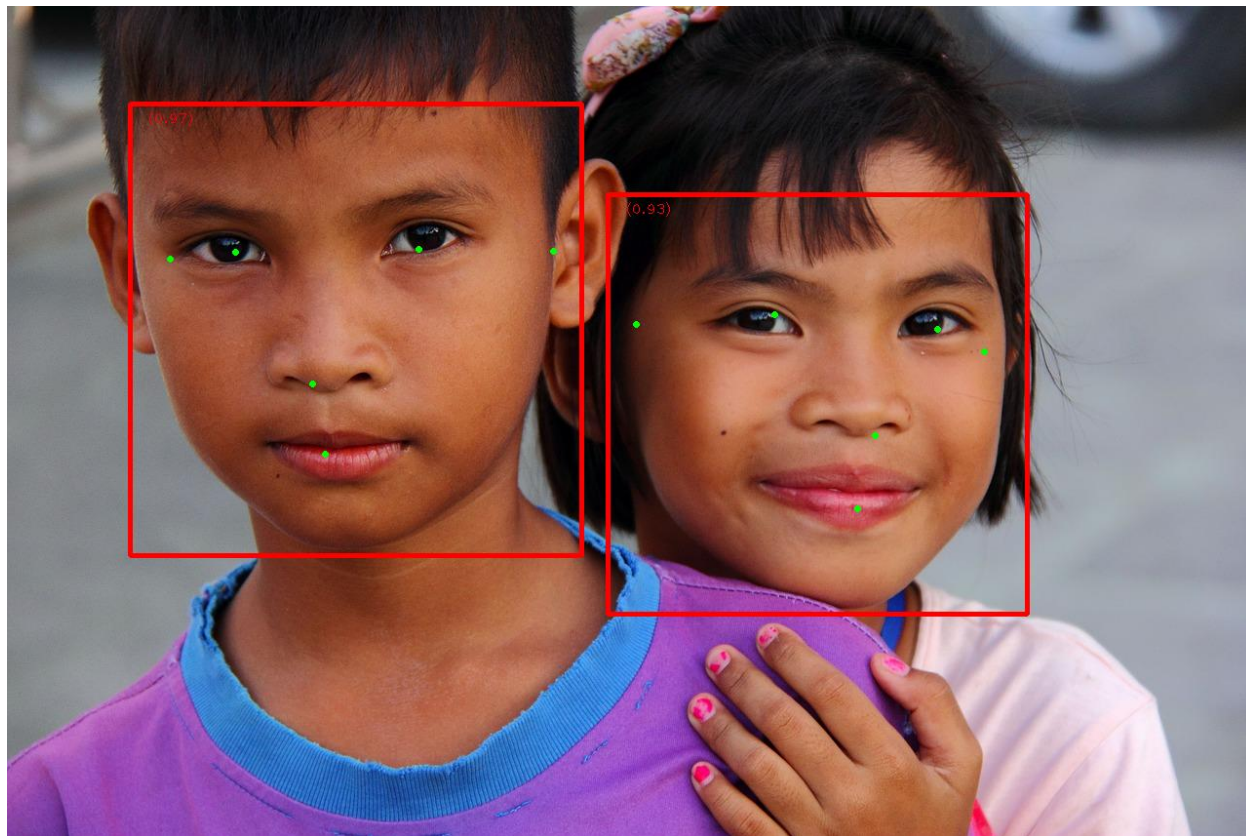
輸出包含
左、右手

- 0. WRIST
- 1. THUMB_CMC
- 2. THUMB_MCP
- 3. THUMB_IP
- 4. THUMB_TIP
- 5. INDEX_FINGER_MCP
- 6. INDEX_FINGER_PIP
- 7. INDEX_FINGER_DIP
- 8. INDEX_FINGER_TIP
- 9. MIDDLE_FINGER_MCP
- 10. MIDDLE_FINGER_PIP
- 11. MIDDLE_FINGER_DIP
- 12. MIDDLE_FINGER_TIP
- 13. RING_FINGER_MCP
- 14. RING_FINGER_PIP
- 15. RING_FINGER_DIP
- 16. RING_FINGER_TIP
- 17. PINKY_MCP
- 18. PINKY_PIP
- 19. PINKY_DIP
- 20. PINKY_TIP

說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker?hl=zh-tw

範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/hand_landmarker/python/hand_landmarker.ipynb

人臉偵測 (Face Detection)



BlazeFace (短距離)	128 x 128	浮點數 16
BlazeFace (全範圍)	128 x 128	float 16
BlazeFace Sparse (全範圍)	128 x 128	float 16

輸出：

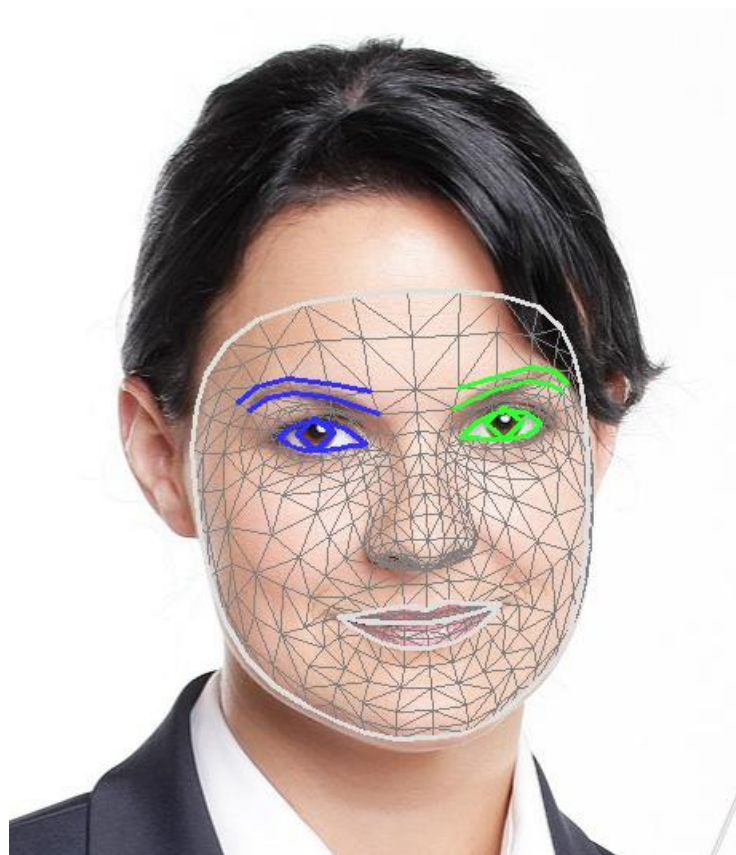
人臉外框、置信度及六個特徵點(Landmark)

- 左眼
- 右眼
- 左耳
- 右耳
- 鼻頭
- 嘴巴

說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/face_detector?hl=zh-tw

範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/face_detector/python/face_detector.ipynb

臉部特徵點偵測 (Face Landmark Detection)



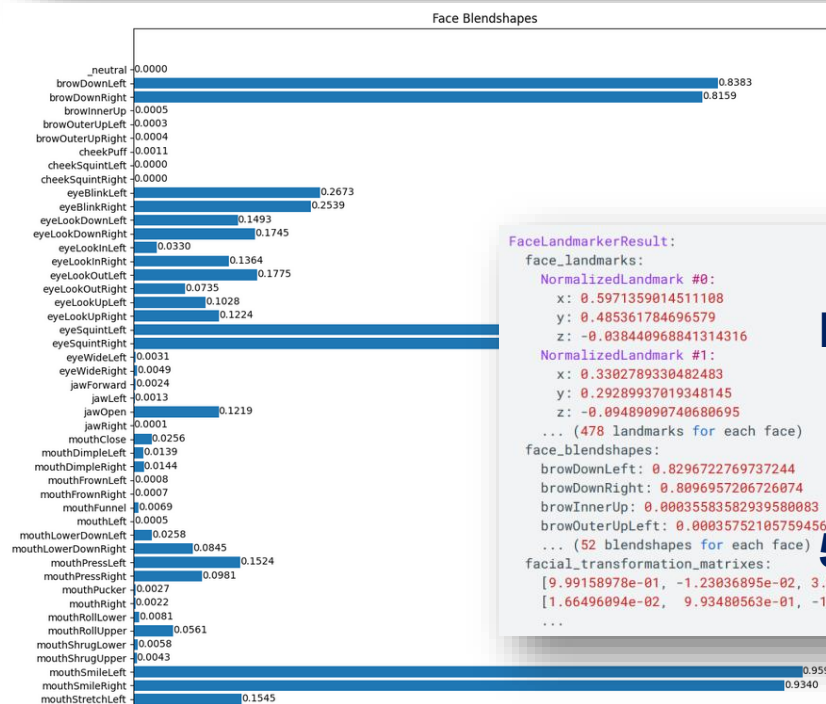
FaceLandmarker

FaceDetector : 192 x 192

float 16

FaceMesh-V2 : 256 x 256

Blendshape : 1 x 146 x 2



混合形狀

FaceLandmarkerResult:

face_landmarks:

NormalizedLandmark #0:

x: 0.5971359814511108

y: 0.485361784696579

z: -0.038440968841314316

NormalizedLandmark #1:

x: 0.3302789330482483

y: 0.29289937819348145

z: -0.09489090740680695

... (478 landmarks for each face)

face_blendshapes:

browDownLeft: 0.8296722769737244

browDownRight: 0.8096957206726074

browInnerUp: 0.00035583582939580083

browOuterUpLeft: 0.00035752105759456754

... (52 blendshapes for each face)

facial_transformation_matrices:

[9.99158978e-01, -1.23036895e-02, 3.91213447e-02, -3.70770246e-01]

[1.66496094e-02, 9.934880563e-01, -1.12779640e-01, 2.27719707e+01]

...

人臉網格

Face Mesh

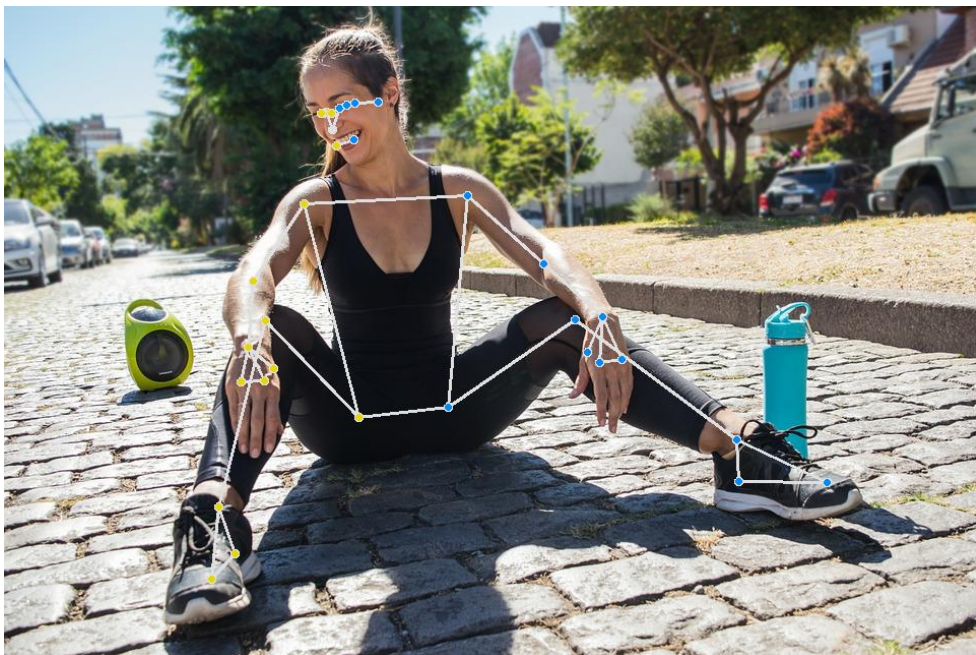
478 特徵點

52 混合形狀

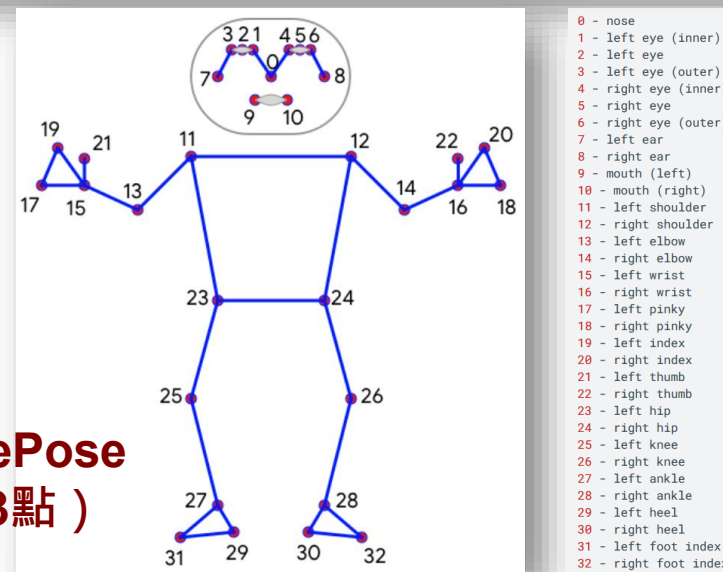
說明文檔 : https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker?hl=zh-tw

範例程式 : https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/face_landmarker/python/%5BMediaPipe_Python_Tasks%5D_Face_Landmarker.ipynb

姿態特徵點偵測 (Pose Landmark Detection)



Pose Landmarker (Lite)	姿勢偵測器：224 x 224 x 3 姿勢標記：256 x 256 x 3	float 16
Pose 地標 (完整版)	姿勢偵測器：224 x 224 x 3 姿勢標記：256 x 256 x 3	float 16
Pose 地標 (大量)	姿勢偵測器：224 x 224 x 3 姿勢標記：256 x 256 x 3	float 16



說明文檔：https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker?hl=zh-tw

範例程式：https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/pose_landmarker/python/%5BMediaPipe_Python_Tasks%5D_Pose_Landmarker.ipynb

整體特徵點偵測 (Holistic Landmark Detection)



輸出總共 543 個地標 (每手可輸出 33 個姿勢地標、468 個臉部地標，以及 21 個手部地標)
目前新版暫不提供，有需要可使用 **Github** 上舊版內容。

綜合範例:浮空手勢簡報播放



說明文檔：<https://omnixri.blogspot.com/2023/05/vmaker-edge-ai-05google-mediapipe.html>

範例程式：https://github.com/OmniXRI/PPT_Gesture_Demo

參考文獻

- 臺灣科技大學資訊工程系 人工智慧與邊緣運算實務 (CS5149701)
<https://omnixri.blogspot.com/p/ntust-edge-ai.html>
- Intel OpenVINO Notebooks Github
https://github.com/openvinotoolkit/openvino_notebooks
- Ultralytics YOLO 說明文檔
<https://docs.ultralytics.com/zh/>
- CMU OpenPose Github
<https://github.com/CMU-Perceptual-Computing-Lab/openpose>
- Google MediaPipe
<https://ai.google.dev/edge/mediapipe/solutions/guide?hl=zh-tw>